

2014年 JOI 本選

問題 1

JOI 紋章

解説担当: 城下 慎也(@phidnight)

最初に

- 本選お疲れ様でした！
- 今から本選の解説が行われます。

問題概要

- 2×2 サイズの JOI 紋章が指定されるので、1 箇所を書き換えを必要ならば実行して、JOI 旗にできるだけ多くの JOI 紋章を含むようにしたときの JOI 紋章の個数の最大値を求める。
- 制約
 - $2 \leq N \leq 1,000$
 - $2 \leq M \leq 1,000$

具体例

- JOI 旗と JOI 紋章の例

JOI 旗

J	O	J	J	O
O	I	J	J	I
J	J	O	I	I
O	I	I	I	J

JOI 紋章

J	O
O	I

具体例

- JOI 旗と JOI 紋章の例

JOI 旗

J	O	J	J	O
O	I	J	J	I
J	J	O	I	I
O	I	I	I	J

JOI 紋章

J	O
O	I

この **J** を

具体例

- JOI 旗と JOI 紋章の例

JOI 旗

J	O	J	J	O
O	I	J	O	I
J	J	O	I	I
O	I	I	I	J

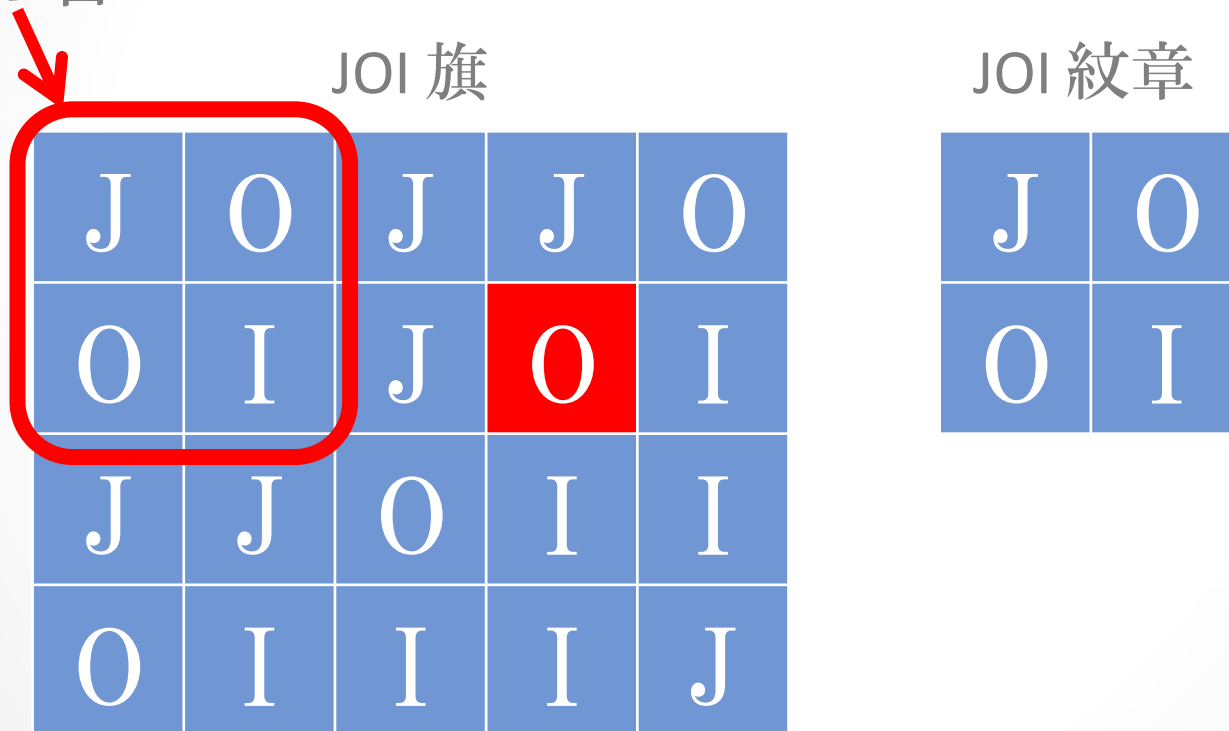
JOI 紋章

J	O
O	I

O にする

具体例

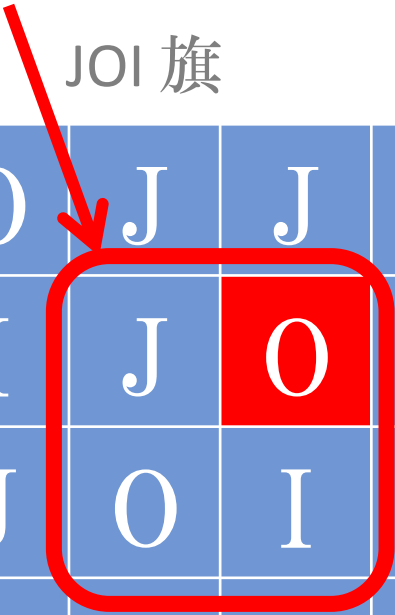
- JOI 旗と JOI 紋章の例
- 1 つ目



具体例

- JOI 旗と JOI 紋章の例
- 1つ目 2つ目

JOI 旗



J	O	J	J	O
O	I	J	O	I
J	J	O	I	I
O	I	I	I	J

JOI 紋章

J	O
O	I

具体例

- JOI 旗と JOI 紋章の例
- 1 つ目 2 つ目 3 つ目

JOI 旗

J	O	J	J	O
O	I	J	O	I
J	J	O	I	I
O	I	I	I	J

JOI 紋章

J	O
O	I

合計 3 個が最大となる。

文字列の問題

- 本問題は文字列を用いた問題である。

文字列の問題

- 本問題は文字列を用いた問題である。
- 文字列の操作には、例えば C++ なら char 配列や string がある。(他の言語にもいろいろある)

文字列の問題

- 本問題は文字列を用いた問題である。
- 文字列の操作には、例えば C++ なら char 配列や string がある。(他の言語にもいろいろある)
- 文字列はエンバグしやすい筆頭
 - バグの原因が文字列操作であるとかは悲しい
 - 貴重なコンテスト時間を失ってしまう

文字列の問題

- 本問題は文字列を用いた問題である。
- 文字列の操作には、例えば C++ なら char 配列や string がある。(他の言語にもいろいろある)
- 文字列はエンバグしやすい筆頭
 - バグの原因が文字列操作であるとかは悲しい
 - 貴重なコンテスト時間を失ってしまう
- 慣れない操作はするべきではない（重要）
 - 事前に構造を把握したもの以外の使用は要注意

文字列の問題

- 本問題は文字列を用いた問題である。
- 文字列の操作には、例えば C++ なら char 配列や string がある。(他の言語にもいろいろある)
- 文字列はエンバグしやすい筆頭
 - バグの原因が文字列操作であるとかは悲しい
 - 貴重なコンテスト時間を失ってしまう
- 慣れない操作はするべきではない（重要）
 - 事前に構造を把握したもの以外の使用は要注意
- 本問題（ここで解説する解法）では特に使用しないが、文字列特有のアルゴリズムとかある。
 - 興味があったら調べてみよう。

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。
- 30点のテストケース→正解
- 70点のテストケース→TLE(時間超過)

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。
- 30点のテストケース→正解
- 70点のテストケース→TLE(時間超過)
→どうして満点が得られないのか？

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。
- 30点のテストケース→正解
- 70点のテストケース→TLE(時間超過)
 - どうして満点が得られないのか？
 - 計算量が $O(N^2 M^2)$ だから！
 - 計算量とは一体？

計算量とは

- プログラムの実行において、比較、代入、加減などの初等動作を行う回数にまつわる関数。

計算量とは

- プログラムの実行において、比較、代入、加減などの初等動作を行う回数にまつわる関数。
- よく $O()$ などの形式で表される。

計算量とは

- プログラムの実行において、比較、代入、加減などの初等動作を行う回数にまつわる関数。
- よく $O()$ などの形式で表される。
- 細かい説明とかは今回省きますが、理解に最低限必要な要点だけまとめると、
 - 実行時間が変数にどう比例するか見積もれる。
 - 主に最高次のものが使われる
 - これによって実行時間を予想できる。
 - 例 : $O(N^2)$, $O(N\log N)$, $O(2^N)$ などなど
 - 値を代入して 10 億超えてたりすると非常に遅い
 - 1 億は切っておきたい(可能なら数百万あたりまで)

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。
- 30点のテストケース→正解
- 70点のテストケース→TLE(時間超過)
 - どうして満点が得られないのか？
 - 計算量が $O(N^2 M^2)$ だから！
 - 計算量とは一体？
 - 制約の式 $N^2 M^2$ に上限(1,000ずつ)を代入すると...

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。
- 30点のテストケース→正解
- 70点のテストケース→TLE(時間超過)
 - どうして満点が得られないのか？
 - 計算量が $O(N^2 M^2)$ だから！
 - 計算量とは一体？
 - 制約の式 $N^2 M^2$ に上限(1,000ずつ)を代入すると...
 - $1000^4 = 10^{12}$ (1 兆くらい！)

部分点解法

- 全てのマス ($N \times M$ マス) について J,O,I それぞれの変更を実行し、変更後の JOI 旗に含まれる JOI 紋章を 1 個ずつ数える。
- 30点のテストケース→正解
- 70点のテストケース→TLE(時間超過)
 - どうして満点が得られないのか？
 - 計算量が $O(N^2 M^2)$ だから！
 - 計算量とは一体？
 - 制約の式 $N^2 M^2$ に上限(1,000ずつ)を代入すると...
 - $1000^4 = 10^{12}$ (1 兆くらい！)
 - これでは間に合わない...

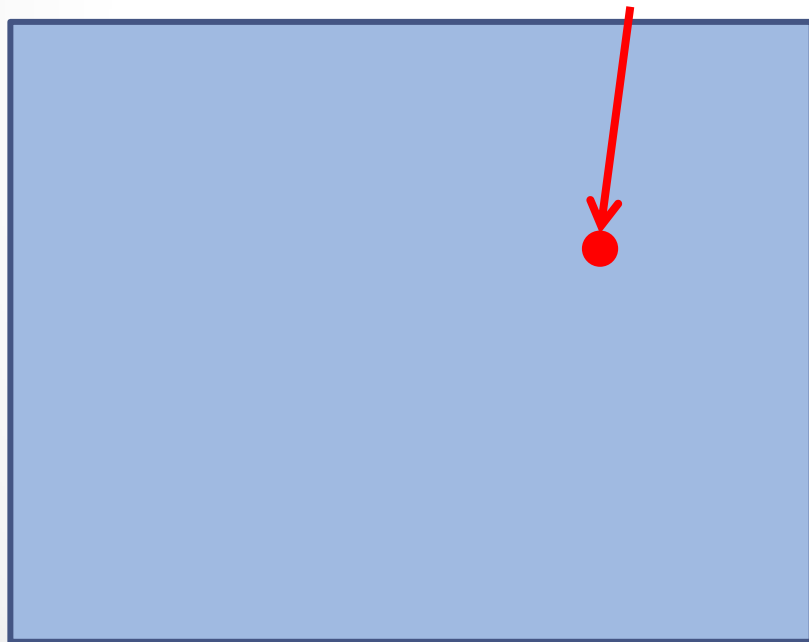
満点解法へのアプローチ

- 部分点解法に何か改善の余地は？

満点解法へのアプローチ

- 部分点解法に何か改善の余地は？
- 実は各書き換えにおいて、わざわざ全部の 2×2 区間を見る必要がない！

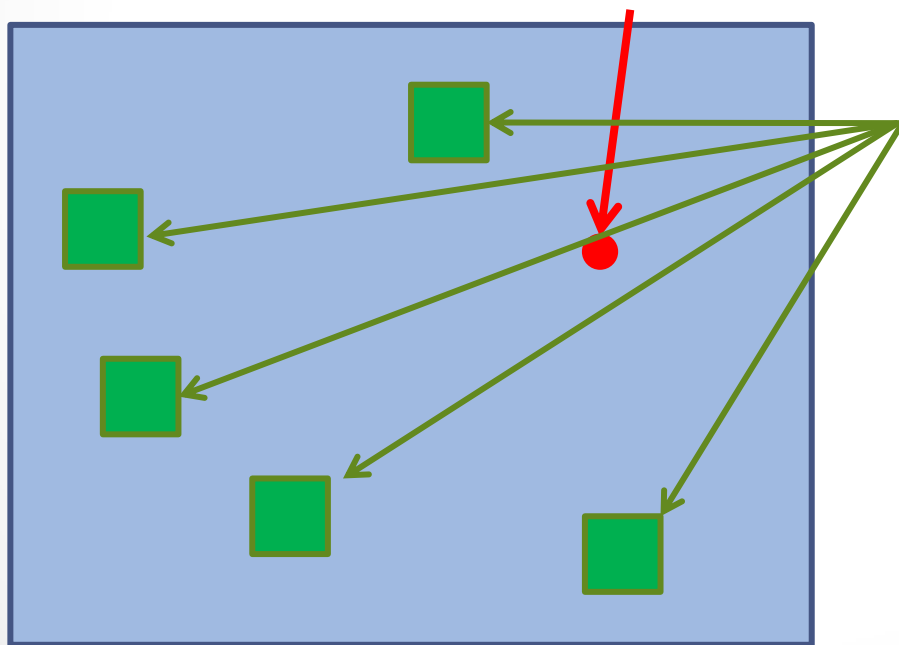
全体に対してここが変化したとき...



満点解法へのアプローチ

- 部分点解法に何か改善の余地は？
- 実は各書き換えにおいて、わざわざ全部の 2×2 区間を見る必要がない！

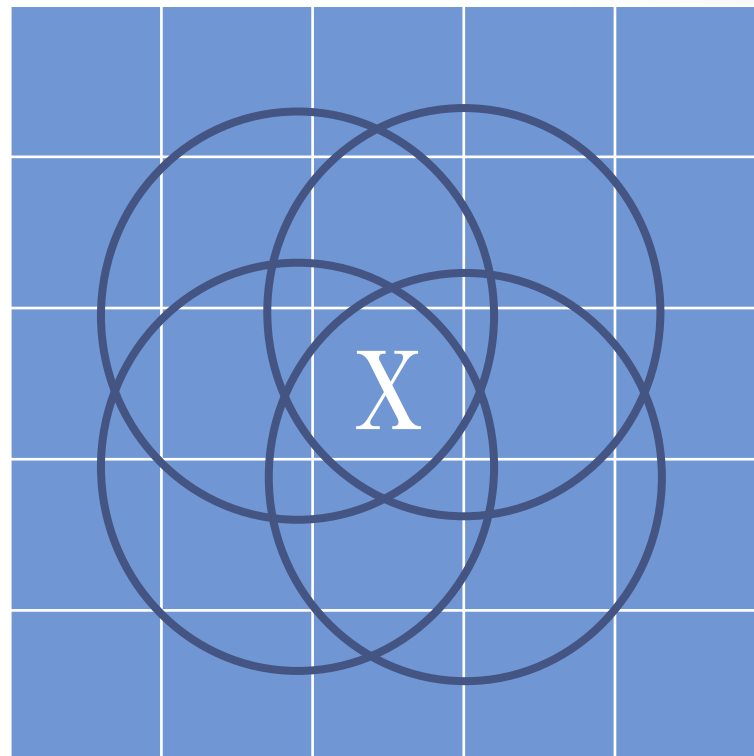
全体に対してここが変化したとき...



こんなところとかは
調べる必要がない！
(書き換えによる
変化がない)

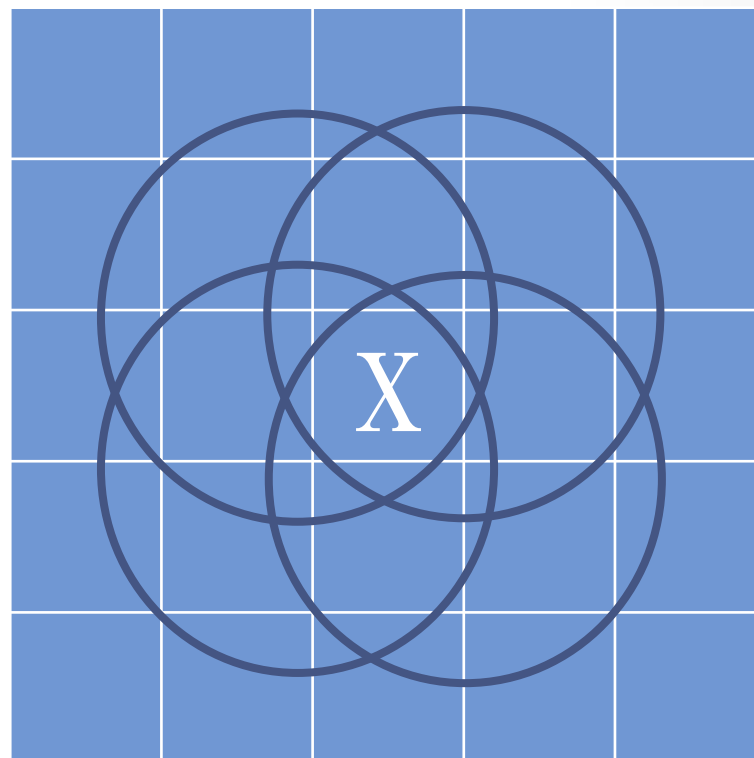
満点解法へのアプローチ

- より細かく言うと、下記のマスXを書き換えた場合にはXを含む4つの場所以外はXの中身によらず同じ結果となっているはず。



満点解法へのアプローチ

- より細かく言うと、下記のマス X を書き換えた場合には X を含む 4 つの場所以外は X の中身によらず同じ結果となっているはず。
- ということは、
(初期状態のJOI 紋章の個数)
+
(書き換えによる変化量)
によって計算できる！



満点解法

- 満点解法は以下の手順で計算できる

満点解法

- 満点解法は以下の手順で計算できる
 - 変更前の JOI 紋章の個数を計算する。
 - 全ての $1 \leq i \leq N, 1 \leq j \leq M$ に対し、次の処理を行う。
 - マス (i, j) を J, O, I それぞれに書き換えてみる
 - 周囲 4 箇所について紋章の増減の最大値を覚える
(増減は -4 から +4 までの範囲の値を取る)
 - 手順2で得られた最大値に手順1の値を加算して答え。

満点解法

- 満点解法は以下の手順で計算できる
 1. 変更前の JOI 紋章の個数を計算する。
 - 全ての領域を探索して $O(NM)$
 2. 全ての $1 \leq i \leq N, 1 \leq j \leq M$ に対し、次の処理を行う。
 - マス (i, j) を J, O, I それぞれに書き換えてみる
 - 周囲 4 箇所について紋章の増減の最大値を覚える
(増減は -4 から +4 までの範囲の値を取る)
 3. 手順2で得られた最大値に手順1の値を加算して答え。

満点解法

- 満点解法は以下の手順で計算できる
 - 変更前の JOI 紋章の個数を計算する。
 - 全ての領域を探索して $O(NM)$
 - 全ての $1 \leq i \leq N, 1 \leq j \leq M$ に対し、次の処理を行う。
 - ・マス (i, j) を J, O, I それぞれに書き換えてみる
 - ・周囲 4 箇所について紋章の増減の最大値を覚える
(増減は -4 から +4 までの範囲の値を取る)
 - 各操作で $O(1)$ となり、手順 2 全体では $O(NM)$ となる。
 - 手順2で得られた最大値に手順1の値を加算して答え。

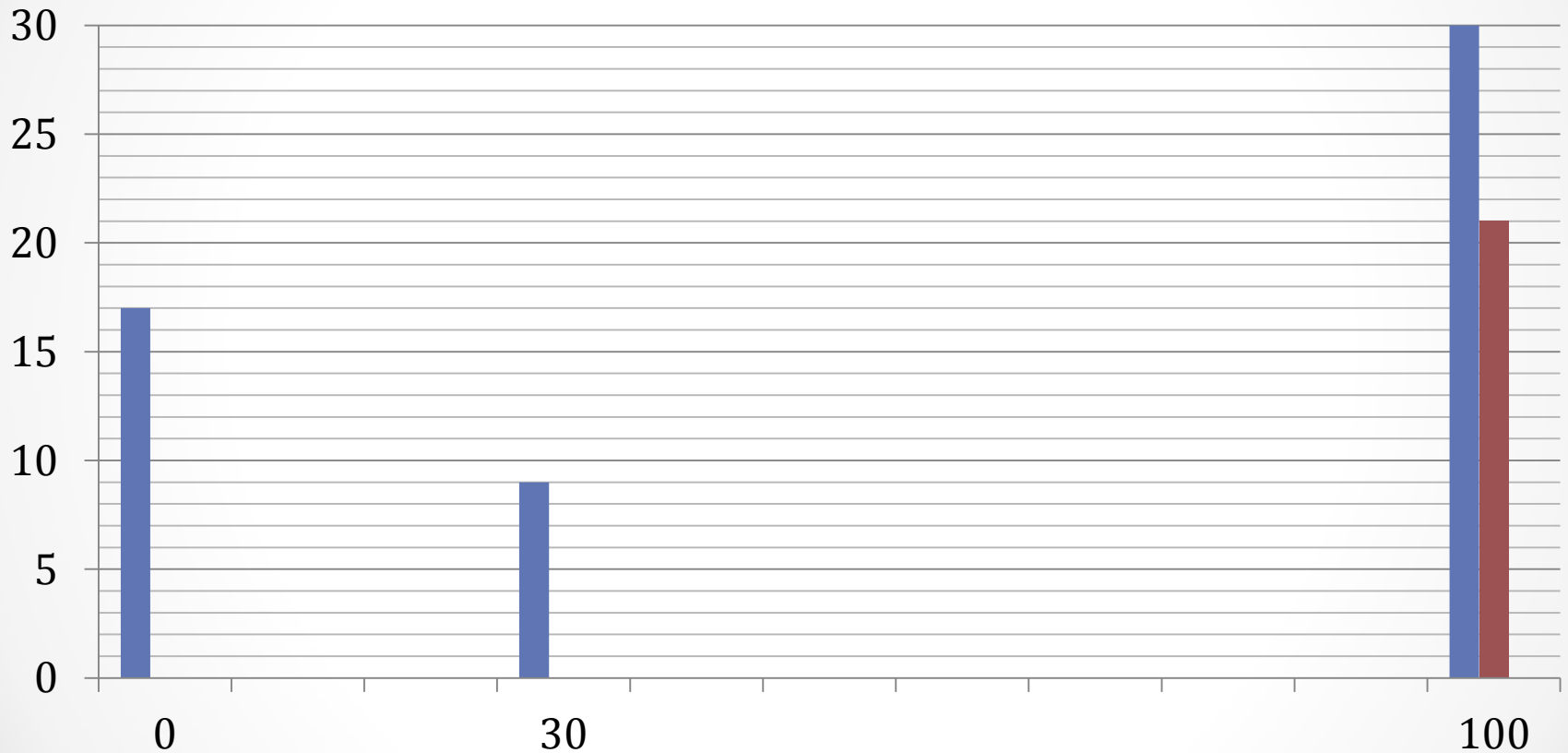
満点解法

- 満点解法は以下の手順で計算できる
 1. 変更前の JOI 紋章の個数を計算する。
 - 全ての領域を探索して $O(NM)$
 2. 全ての $1 \leq i \leq N, 1 \leq j \leq M$ に対し、次の処理を行う。
 - マス (i, j) を J, O, I それぞれに書き換えてみる
 - 周囲 4 箇所について紋章の増減の最大値を覚える
(増減は -4 から +4 までの範囲の値を取る)
 - 各操作で $O(1)$ となり、手順 2 全体では $O(NM)$ となる。
 3. 手順2で得られた最大値に手順1の値を加算して答え。
 - 全体で $O(NM)$ となる。

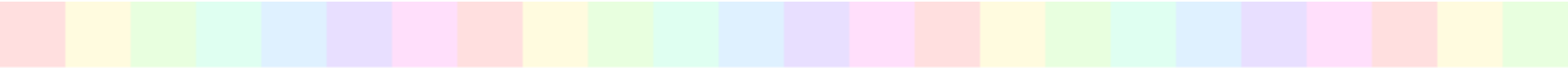
満点解法

- 満点解法は以下の手順で計算できる
 1. 変更前の JOI 紋章の個数を計算する。
 - 全ての領域を探索して $O(NM)$
 2. 全ての $1 \leq i \leq N, 1 \leq j \leq M$ に対し、次の処理を行う。
 - マス (i, j) を J, O, I それぞれに書き換えてみる
 - 周囲 4 箇所について紋章の増減の最大値を覚える
(増減は -4 から +4 までの範囲の値を取る)
 - 各操作で $O(1)$ となり、手順 2 全体では $O(NM)$ となる。
 3. 手順 2 で得られた最大値に手順 1 の値を加算して答え。
 - 全体で $O(NM)$ となる。
 - N, M に 1000 を代入しても 100 万ほどで高速(満点)

得点分布



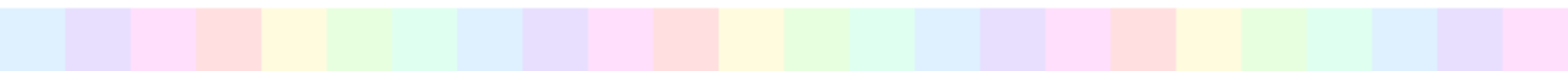
- 赤色は 31 人目以降を表しています。



IOI 饅頭 (IOI Manju)

JOI 2014 本選 問題 2

解説： 保坂 和宏



問題概要

- M 個の饅頭
 - 価格 P_i
- N 個の箱
 - 入る饅頭の個数 C_j
 - 価格 E_j
- 箱をいくつか買って、饅頭を詰めて売るとき、利益の最大値は？
- $M \leq 10000, N \leq 500$

問題概要

¥ 180

¥ 160

¥ 170

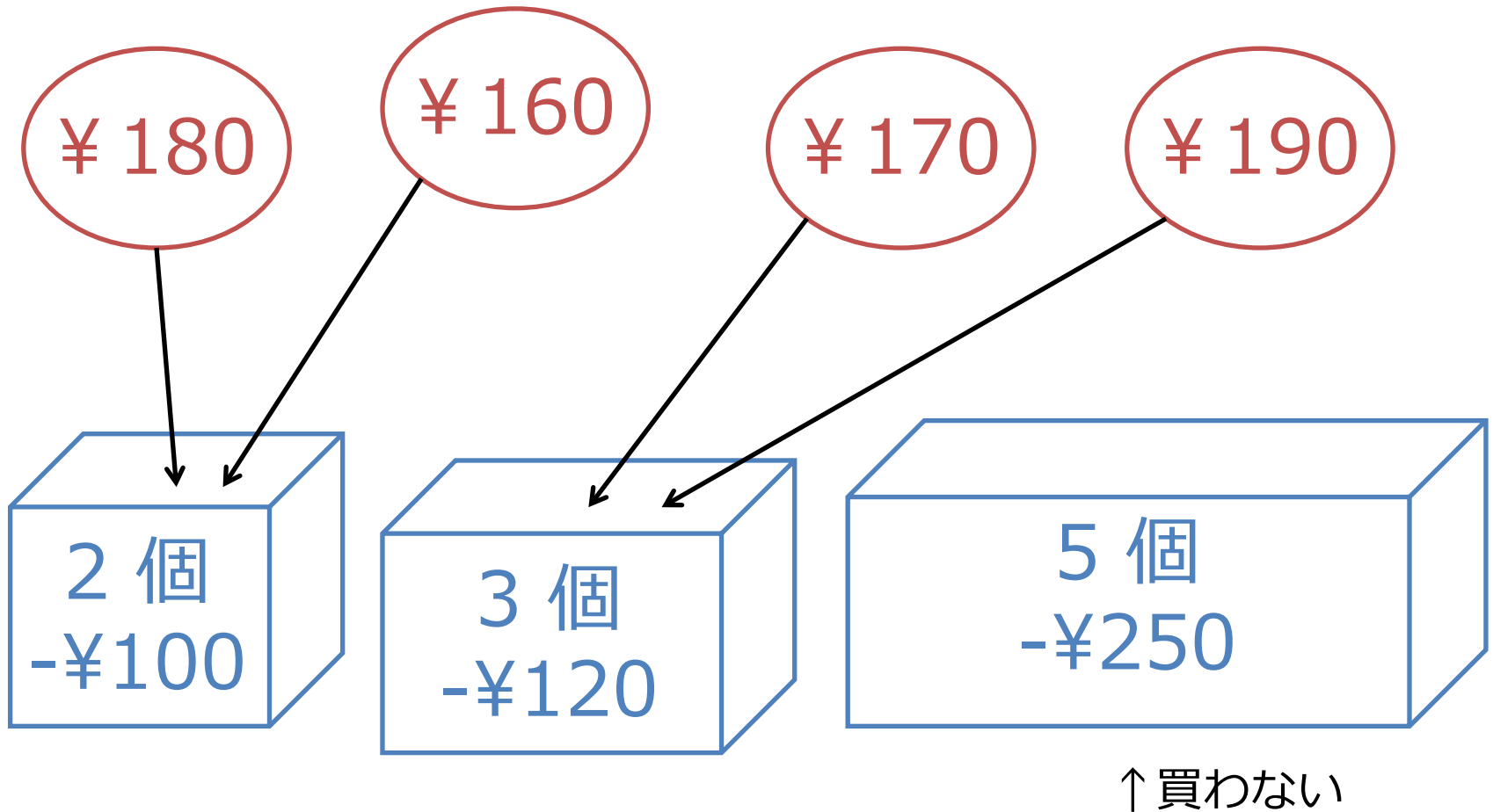
¥ 190

2 個
-¥100

3 個
-¥120

5 個
-¥250

問題概要



考察

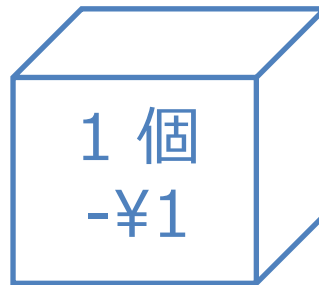
- 同じ個数の饅頭を売るなら，高い方から順に売るほうがよい

¥ 1000

¥ 100

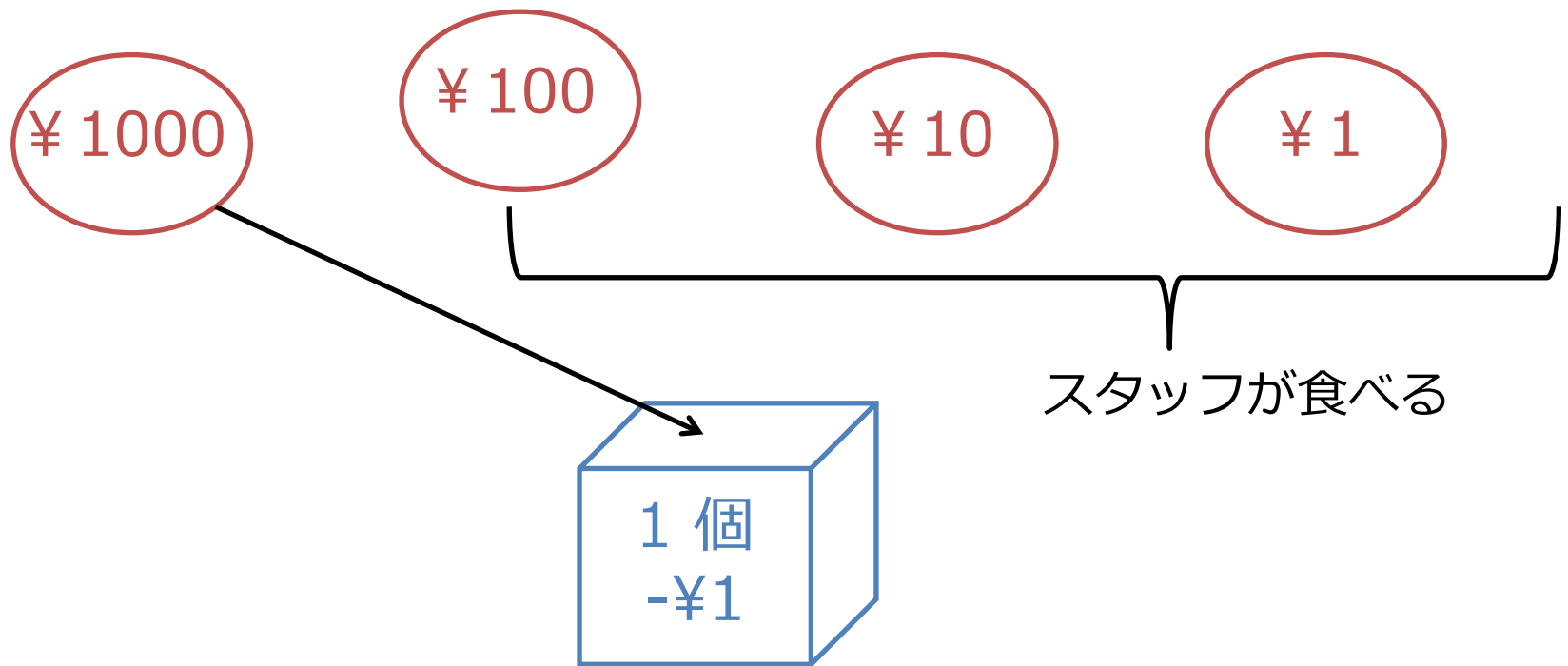
¥ 10

¥ 1



考察

- 同じ個数の饅頭を売るなら，価格が高い方から順に売るほうがよい



部分点解法

- 小課題 1 [25 点] : 箱の個数 $N \leq 10$
- 買う箱の組合せを決める (2^N 通り試す)
- 売れる饅頭の個数の最大値 x が決まる
- 価格が高い方から x 個の饅頭を売る

考察

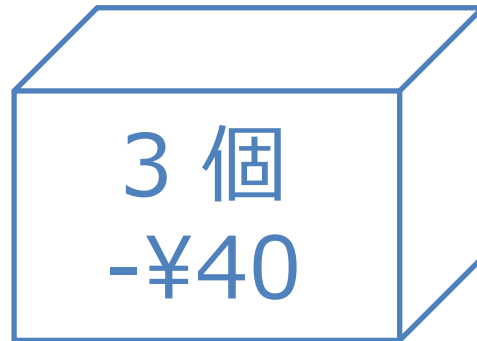
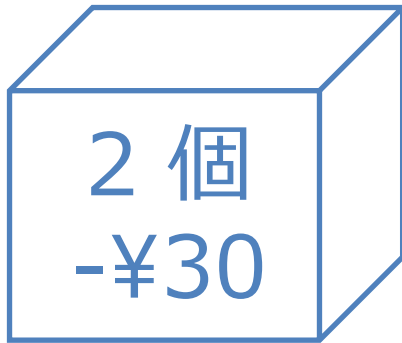
- 買う箱の組合せを決める (2^N 通り試す)
- 売れる饅頭の個数の最大値 x が決まる
- 価格が高い方から x 個の饅頭を売る

考察

- 饅頭の個数を決めたら？
 - 売上：簡単に求まる
 - 箱にかかる費用：？？？
- 「 x 個以上の饅頭が入るようにするための箱の価格の合計」を $x = 0, 1, 2, \dots, M$ に対して効率よく求めたい

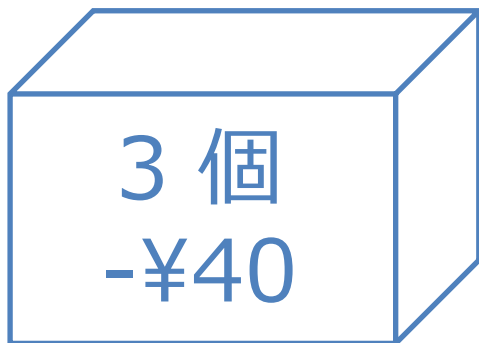
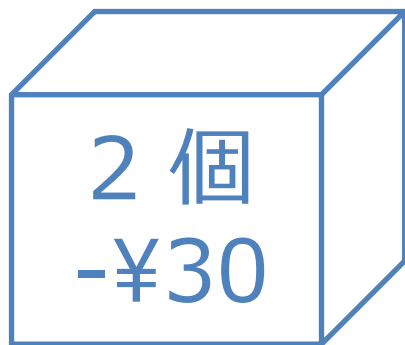
[illegible]

箱が 2 個の場合



0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞

箱が 3 個の場合

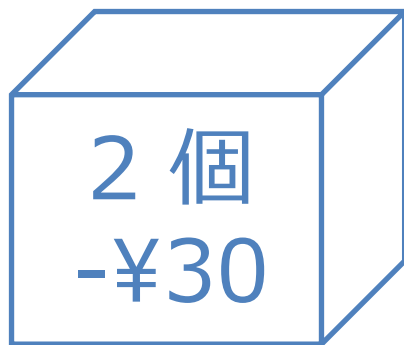


0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

考察

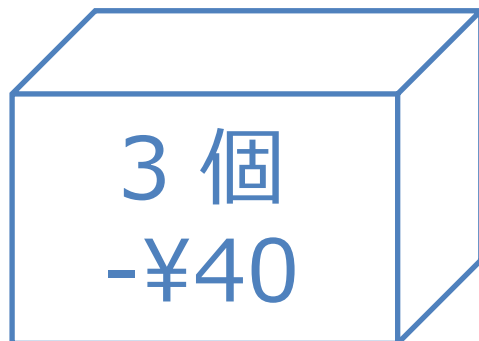
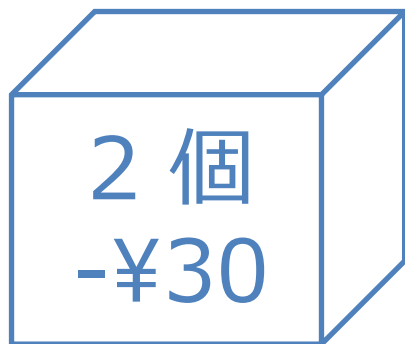
- 「 x 個以上の饅頭が入るようにするための箱の価格の合計」を $x = 0, 1, 2, \dots, M$ に対して効率よく求めたい
- 箱が 1 個少ない場合に求まっていれば表を更新できる

箱が 3 個の場合



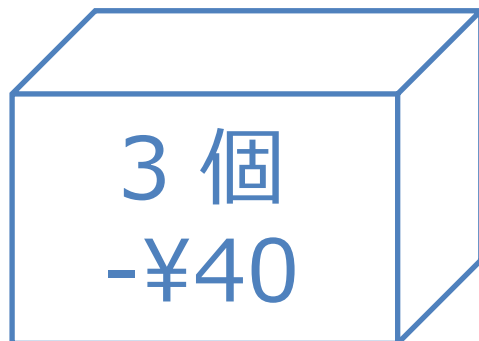
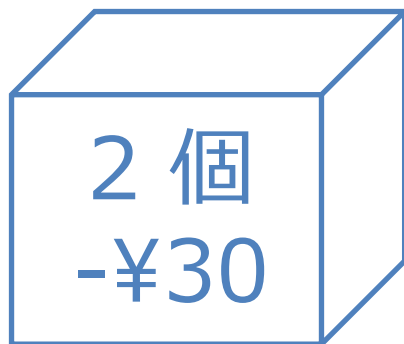
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



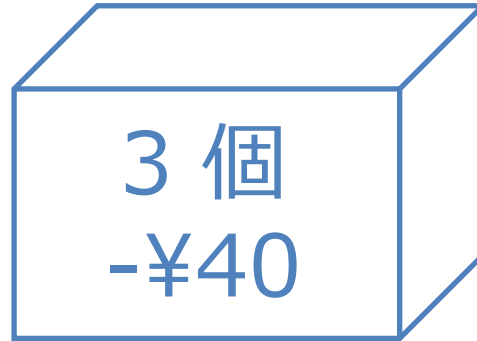
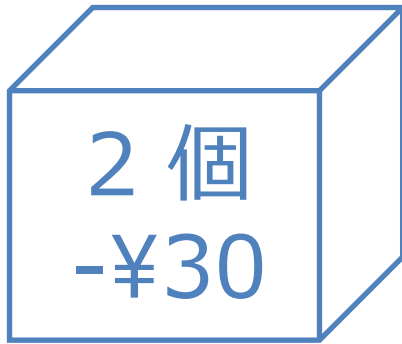
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



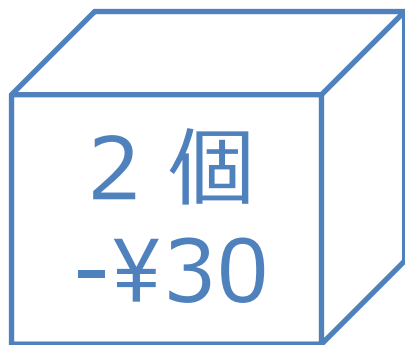
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



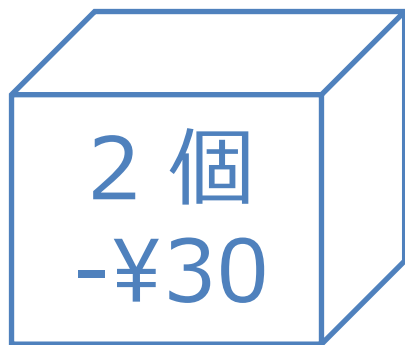
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



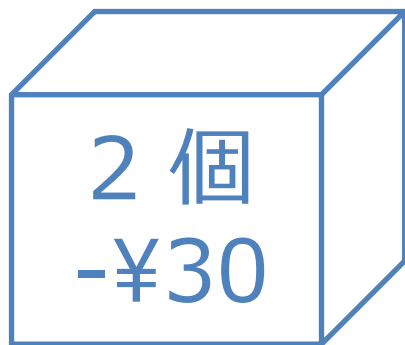
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



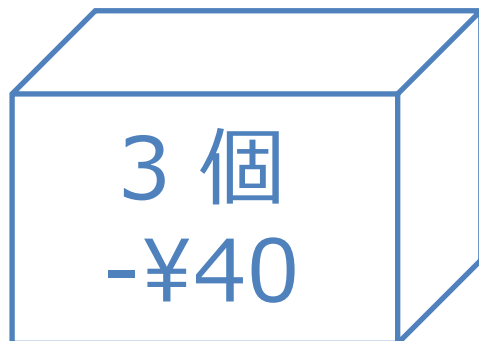
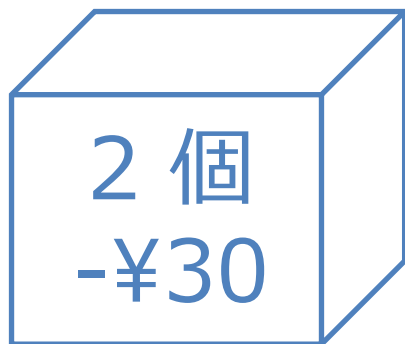
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



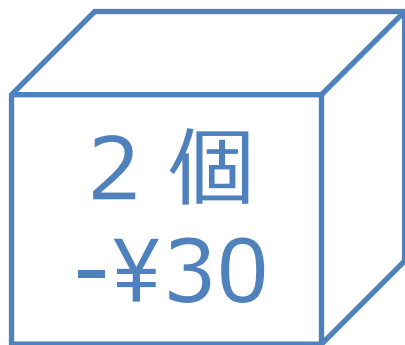
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



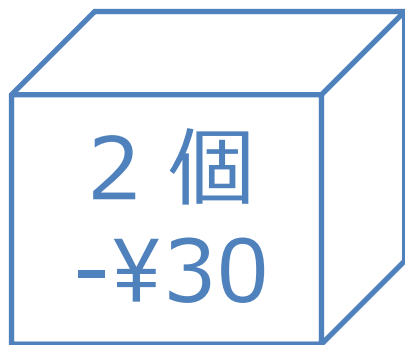
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



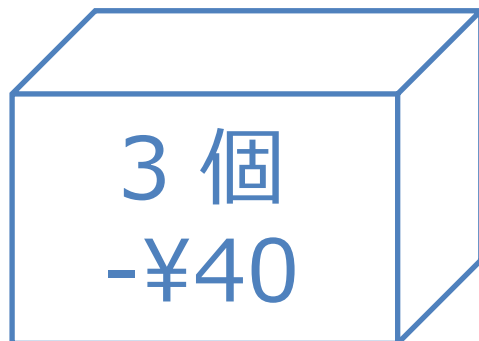
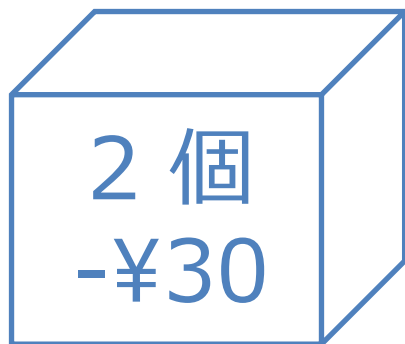
0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

箱が 3 個の場合



0	1	2	3	4	5	6	7	8	9	10
0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	∞	∞	∞	∞	∞	∞	∞	∞
0	30	30	40	70	70	∞	∞	∞	∞	∞
0	30	30	40	50	50	80	80	90	120	120

解法

- $f[j, x] := j$ 番目の箱まで考えたときの,
 x 個以上の饅頭が入るようにするための箱
の価格の合計
- $f[j, x]$ は次のうち小さい方
 - $f[j - 1, x]$
 - $f[j - 1, \max\{j - C_j, 0\}] + E_j$

解法

- $M \times N$ の表を埋める
- 各マスを埋めるのに $O(1)$ 時間
- 全体で $O(M N)$ 時間
- 小さい問題に対して解いて、表を埋めていく：**動的計画法**と呼ばれる

部分点解法

- 小課題 2 [35 点] : 箱の大きさ $C_j \leq 10$
- 表の 1 マスを埋めるのに $O(C_j)$ 時間
 - 箱を大きさ $0, 1, 2, \dots, C_j$ として使う場合をそれぞれ考える
- その他バグなど
 - 箱の大きさが余る場合にも大丈夫ですか？

おまけ

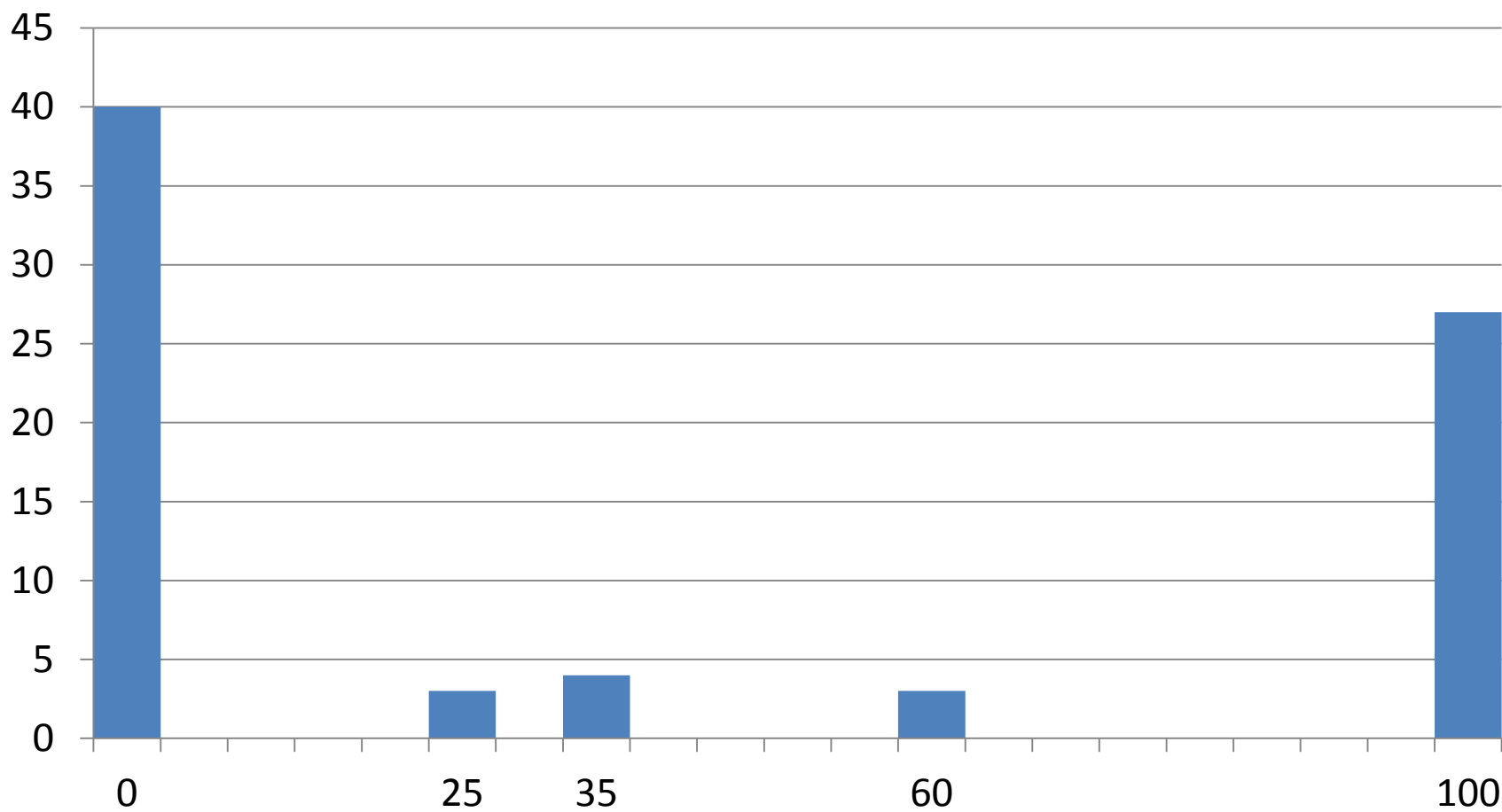
- JOI 社は箱の販売で利益を得ている？！

あなたは Just Odd Inventions 社を知っているだろうか？ この会社の業務は「ただ奇妙な発明 (just odd inventions)」をすることである。ここでは略して JOI 社と呼ぶ。

なお、JOI 社が「ただ奇妙な発明」をすることでどうやって利益を得ているかは、社内でも最高機密であり社長以外の誰も知らない。

JOI 2009 本選 5 「認証レベル」

得点分布



2013-2014年JOI本選問題第3問

バームクーヘン



笠浦一海

問題



- バームクーヘンを三つに分ける
- 切れるところは決まってる
- 三つのうち大きさが最小のピースの大きさを最大化したい

愚直な方法



- 全探索
- 切る部分を決める→約 $N^3/6$ 通り
- それぞれのピースの大きさを求める
→愚直に合計すると $O(N)$
- 合わせて $O(N^4)$
- Subtask1 $N \leq 100$ なら解ける→5点

注意点



- 各部分のおおきさ A_i の条件
- $1 \leq A_i \leq 10000000000$
- intの範囲ぎりぎり
- **オーバーフローに注意**
- long longを使うべき

注意点



- 円環上の数列を扱う。
- A の後ろに A のコピーをつなげて
- $A_{N+1}=A_1, \dots, A_{N+i}=A_i, \dots, A_{2N}=A_N$
- とすると実装上便利

注意点



- 切れ込み1を含むようなピースの大きさ
- $A_i \dots A_N, A_1 \dots A_j$ の合計を求めるとき

```
long long Sum=0;
```

```
for(int k=i;k<=N;k++)
```

```
    Sum+=A[k];
```

```
for(int k=1;k<=j;k++)
```

```
    Sum+=A[k];
```


注意点



- 切れ込み1を含むようなピースの大きさ
- $A_i \dots A_N, A_1 \dots A_j$ の合計を求めるとき

```
long long Sum=0;
```

```
for(int k=i;k<=j+N;k++)  
    Sum+=A[k];
```

もう少しましな方法



- 前の方法ではいちいち合計を求めている。
- 同じ区間について何度も合計を求めている無駄。
- 区間の数 $O(N^2)$ 個についてあらかじめ合計を求めて記憶しておく。
- 前処理 $O(N^3)$ で、それぞれの切り方についてしらべる段階でも $O(N^3)$
- 全体計算量 $O(N^3)$
- Subtask2 $N \leq 400$ までならOK → 20点

もう少しましな方法2



- 累積和
- $A_1 \dots A_i$ の合計を B_i とおく。
- 漸化式
 - $B_0 = 0$
 - $B_{i+1} = B_i + A_{i+1}$
- で $O(N)$ で計算可能。

もう少しましな方法2



● 累積和

- $A_i \dots A_j$ の合計 $= B_j - B_{i-1}$
- $A_i \dots A_N, A_1 \dots A_j$ の合計 $= B_N - B_{i-1} + B_j = B_{j+N} - B_{i-1}$
- と各ピースの大きさを $O(1)$ で求められる。
- これでも計算量 $O(N^3)$ になる。
- 以下のスライドでも区間の合計は定数時間で求められるとする

解法



- これ以上の点数を取るのに必要なテクニック
- 二分探索
- 尺取り法

解で二分探索



- ①「3つのピースの大きさの最小値の最大値を求めよ」
- という問題を次の判定問題に還元する
- ②「3つのピースとも大きさがX以上になる切り方は存在するか？」
- ②が解けたとする

解で二分探索



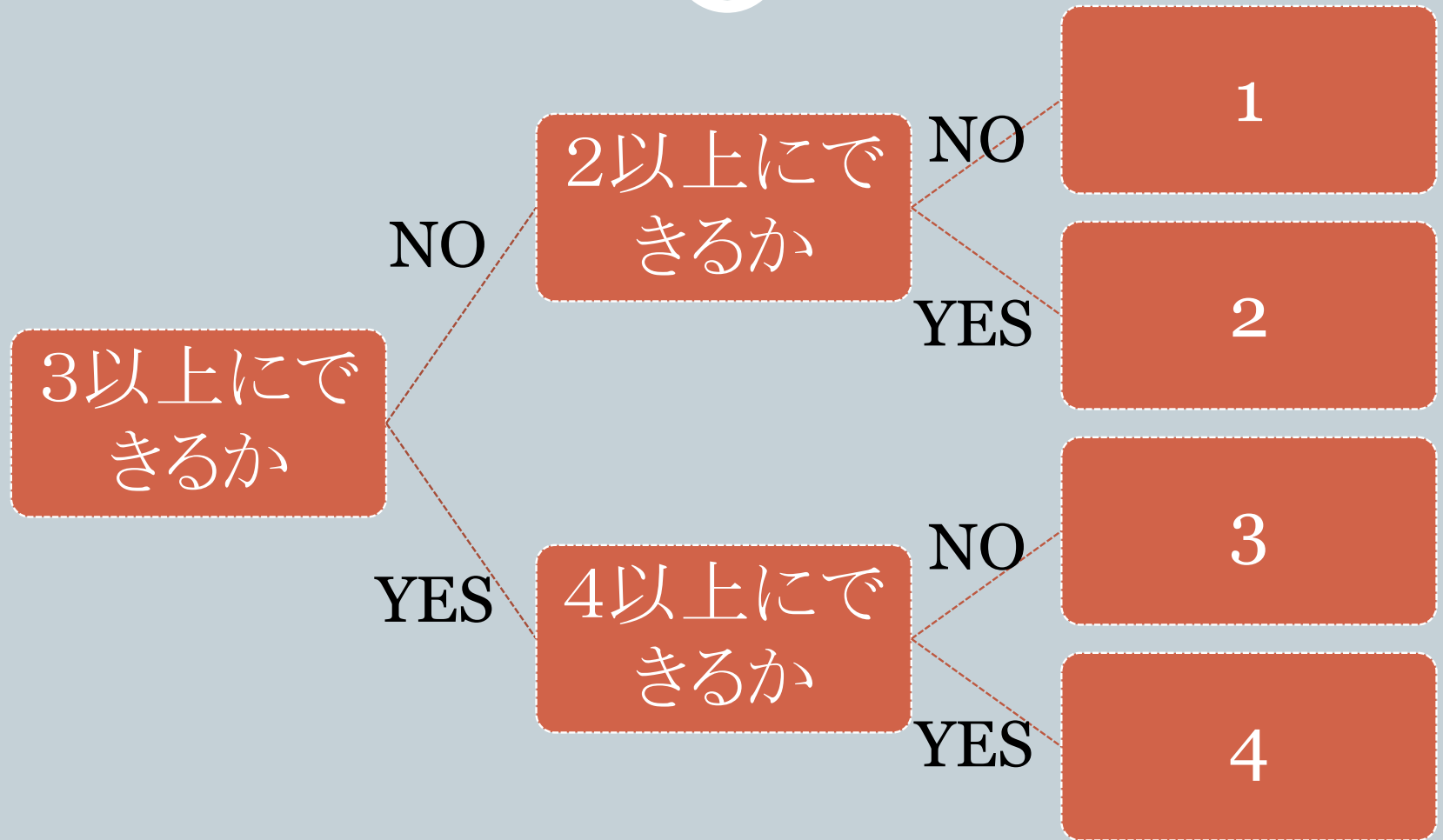
- 求めたい最大値の上限を適当に定めてMAXとする。
- この問題では $\text{Max} = 10000000000 * N$ とすればよい。
- 最大値は $[1, \text{Max} + 1)$ の範囲に存在。
- $m = (\text{Max} + 2) / 2$ とする。
- $X = m$ について②を解く。
- Yes → 求める値は $[m, \text{Max} + 1)$ の範囲に存在。
- No → 求める値は $[1, m)$ の範囲に存在。

解で二分探索



- これを再帰的に行う。
- 求めたい値が $[l, r)$ の範囲に存在することが分かっているとする。
- $m = (l + r) / 2$ とする。
- $X = m$ について②を解く。
- Yes → 求める値は $[m, r)$ の範囲に存在。
- No → 求める値は $[l, m)$ の範囲に存在。
- $r - l = 1$ となるまで繰り返す。

例(Max=4)



解で二分探索

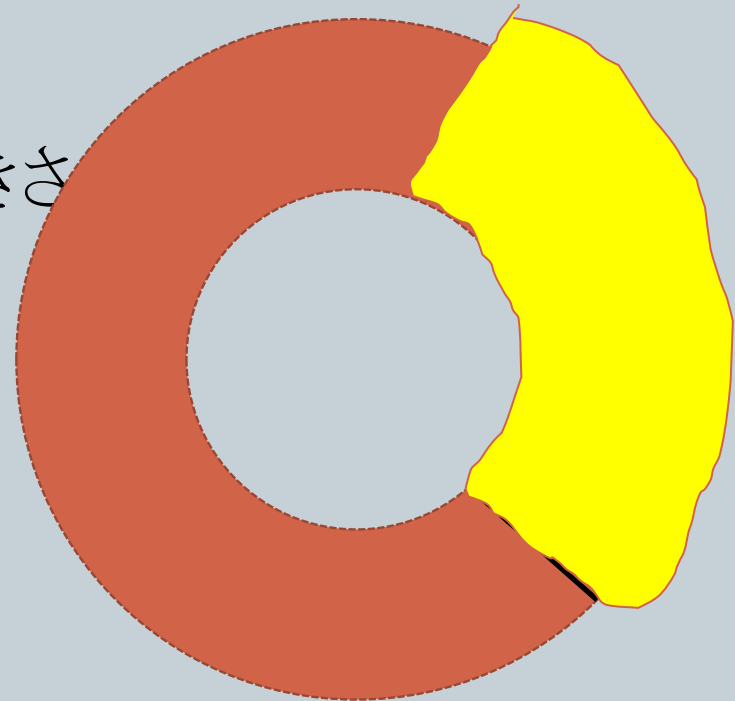


- すなわち②が解ければ①は
- $(\text{②を解くのにかかる時間}) \times \log \text{Max}$
- で解くことができる。

②の解き方



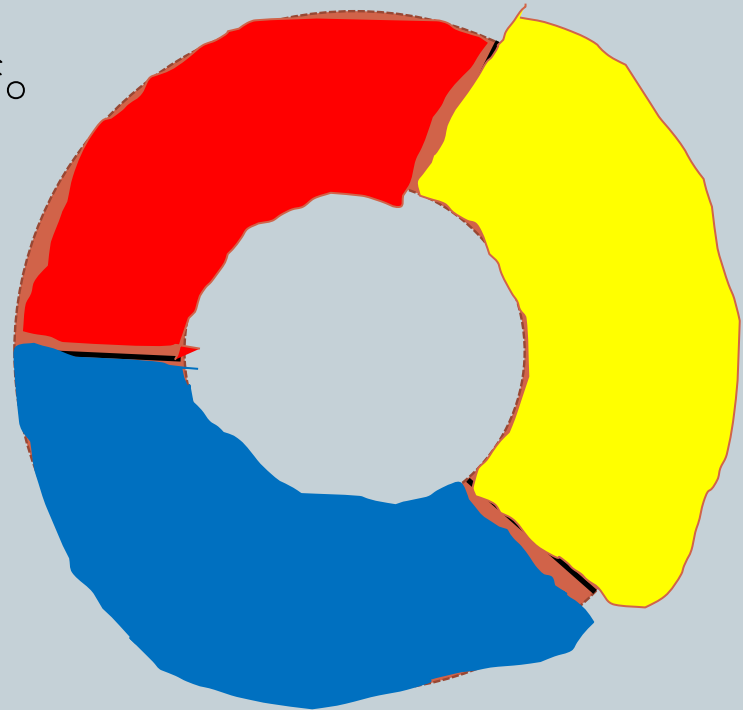
- 「どのピースも大きさが X 以上になるように切ることができるか？」
- まず切る位置を一つ決める。
- 次に切る位置を決めたい。
- 両者の間のピース(黄)の大きさが X 以上ならよい
- 逆に X 以上ならそれより大きくしても無意味



②の解き方



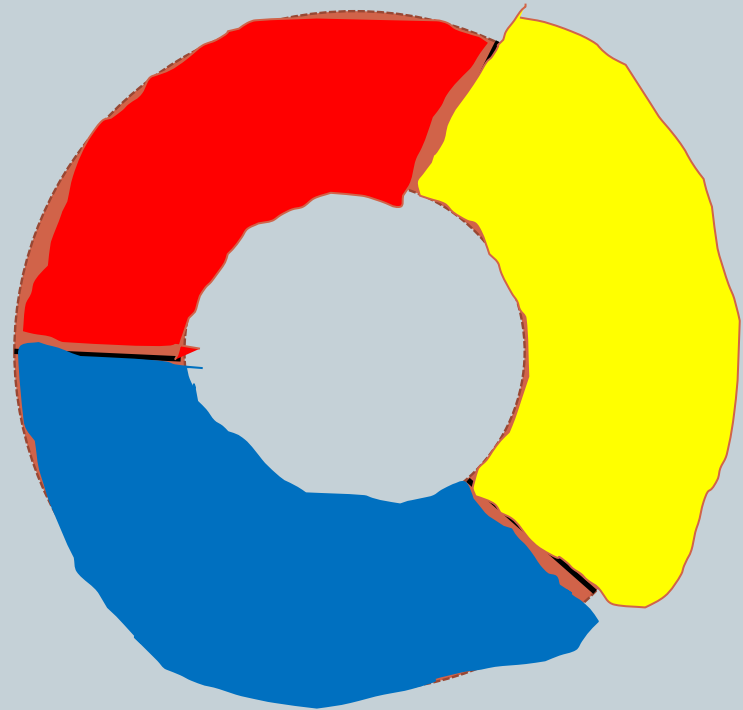
- 「どのピースも大きさが X 以上になるように切ることができるか？」
- 間の部分の大きさが最初に X を超える位置できるのが最善。順番に見ていけばそのような位置は $O(N)$ で見つかる。
- もう1か所も同様に切る。
- 残りの部分(赤色)の大きさが X 以上なら成功。



②の解き方



- 「どのピースも大きさがX以上になるように切ることができるか？」
- 切る位置N通りについて調べる。
- どれか一つで成功ならYes
- 成功がないならNo
- ②が $O(N^2)$ で解ける。
- ①は $O(N^2 \log \text{Max})$
- Subtask 3 $N \leq 8000$
は少し厳しい。



②の早い解き方(二分探索)



- 「どのピースも大きさがX以上になるように切ることができるか？」
- 「最初にXを超える位置」を二分探索で求める。
- 求めたいのは

$$B_l - B_{i-1} \geq X$$

となるlの最小値

- 自分で二分探索を書かなくてもlower_boundが使える。

②の早い解き方(二分探索)



- 「どのピースも大きさがX以上になるように切ることができるか？」
- こうすると $O(N \log N)$ で②が解ける
- ①の計算量は $O(N \log N \log \text{Max})$ となる。
- $N \leq 1000000$ なら大丈夫→満点！

②の早い解き方(尺取り法)

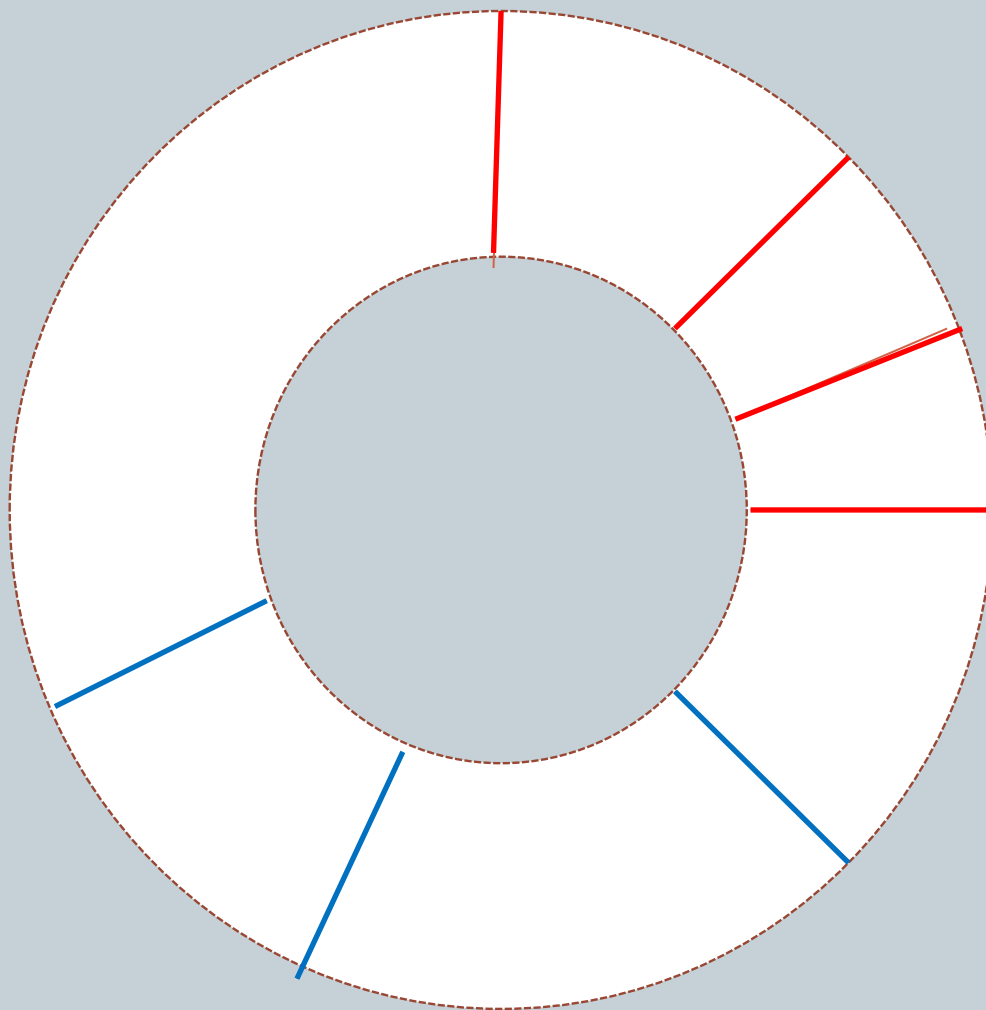


- 「どのピースも大きさが X 以上になるように切ることができるか？」
- i について $B_1 - B_{i-1} \geq X$ なる l の最小値を $l(i)$ と書く。
- $l(i+1) \geq l(i)$
- なぜならば $B_{l(i)-1} - B_i \leq B_{l(i)-1} - B_{i-1} < X$ より $l(i)-1 < l(i+1)$
- $l(i+1)$ を調べるときは $l(i)$ から見ていけばよい。
- これで実は $O(N)$ になる。

②の早い解き方(尺取り法)



$$X=135^{\circ}$$



②の早い解き方(尺取り法)

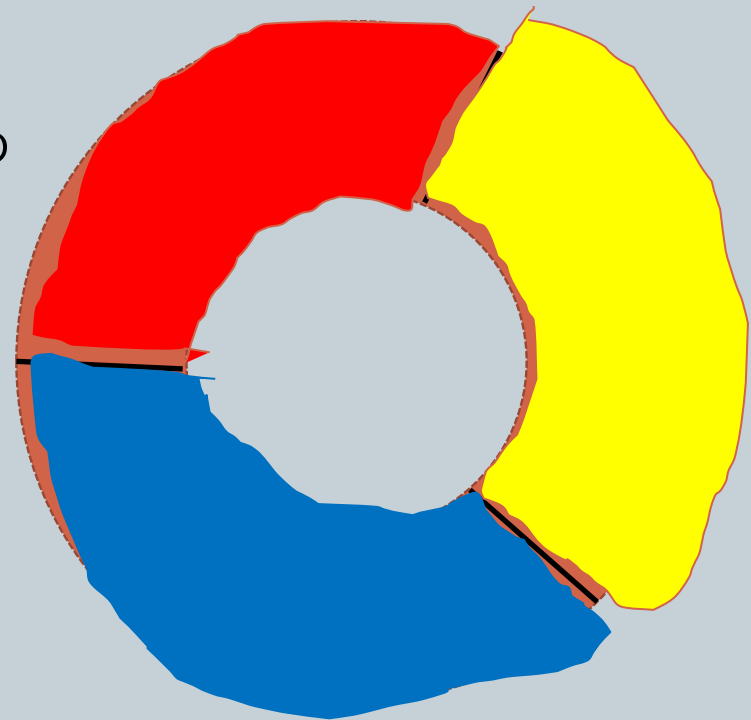


- 尺取り虫っぽい動きをしている→これを尺取り法という
- $l(i)$ が単調に増加するので値が増える回数は高々 $2N$
- $O(N)$ ですべての i に対する $l(i)$ 計算ができる。
- $l(i)$ を計算しておけば②が $O(N)$ で解ける。
- ①が $O(N \log \text{Max})$ で解ける→満点

別解



- 「ある区間について、それが最小のピースとなりうるか？」という問題を考える。
- その区間(黄色)のサイズを X とする。
- もうひとつのピース(青色)の部分の大きさが X 以上になる最初ところで切る
- `lower_bound`が使える。
- 残った赤色のピースの大きさが X 以上なら可能。
- そうでないなら不可能。



別解



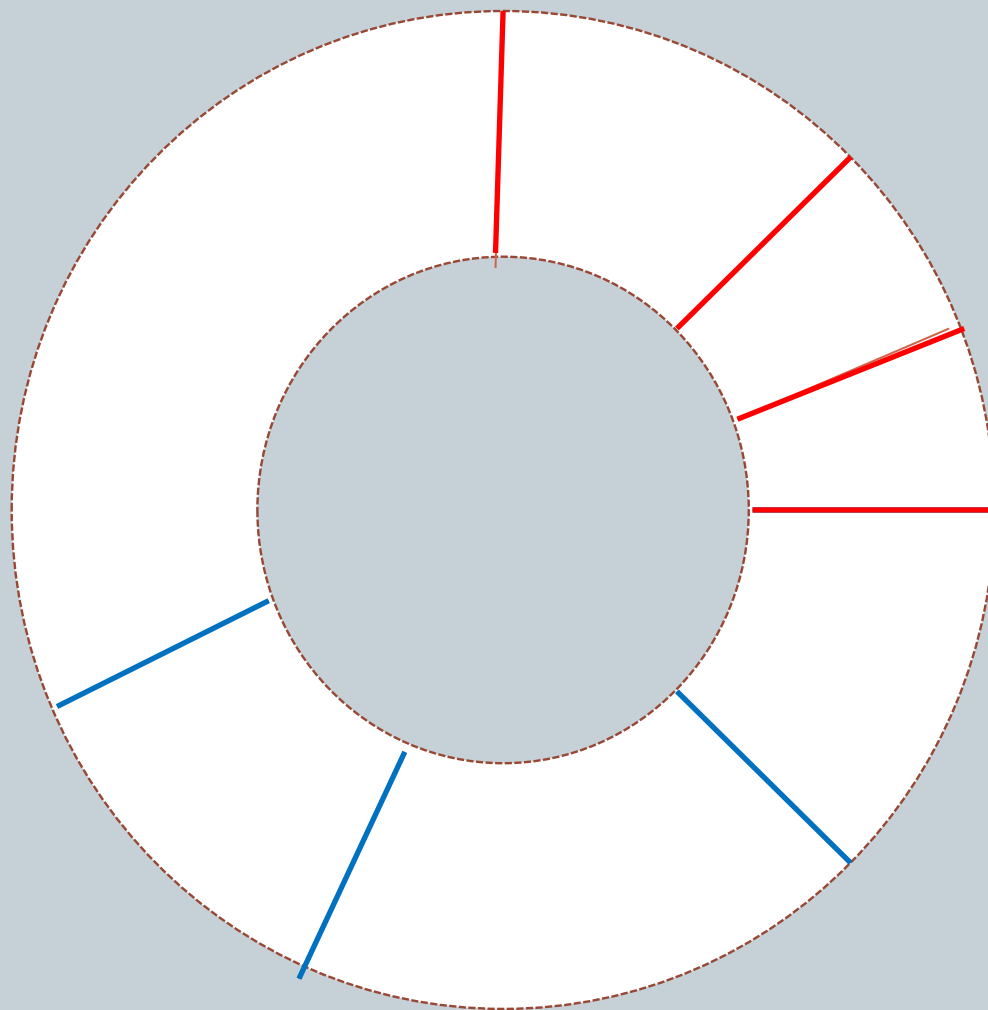
- すべての区間について調べて、最小のピースとなりえるものの最大値を取る。
 - 計算量は $O(N^2 \log N)$
 - 定数が軽いので適当に枝刈りすれば
- Subtask 3: $N \leq 8000$ もぎりぎり通りそう。
- 50点?

別解



- 各 i について $A_i \sim A_m$ の区間が最小のピースになりうるような l の最大値を $m(i)$ とする。
 - 求めたいのは $B_{m(i)} - B_{i-1}$ の最大値。
 - $m(i+1) \geq m(i)$ となる。
- 先ほどと同様に尺取り法が使える！
- 条件を満たすかの判定に $O(\log N)$ かかるので、全体の計算量は $O(N \log N)$ になる。
- 満点！

別解

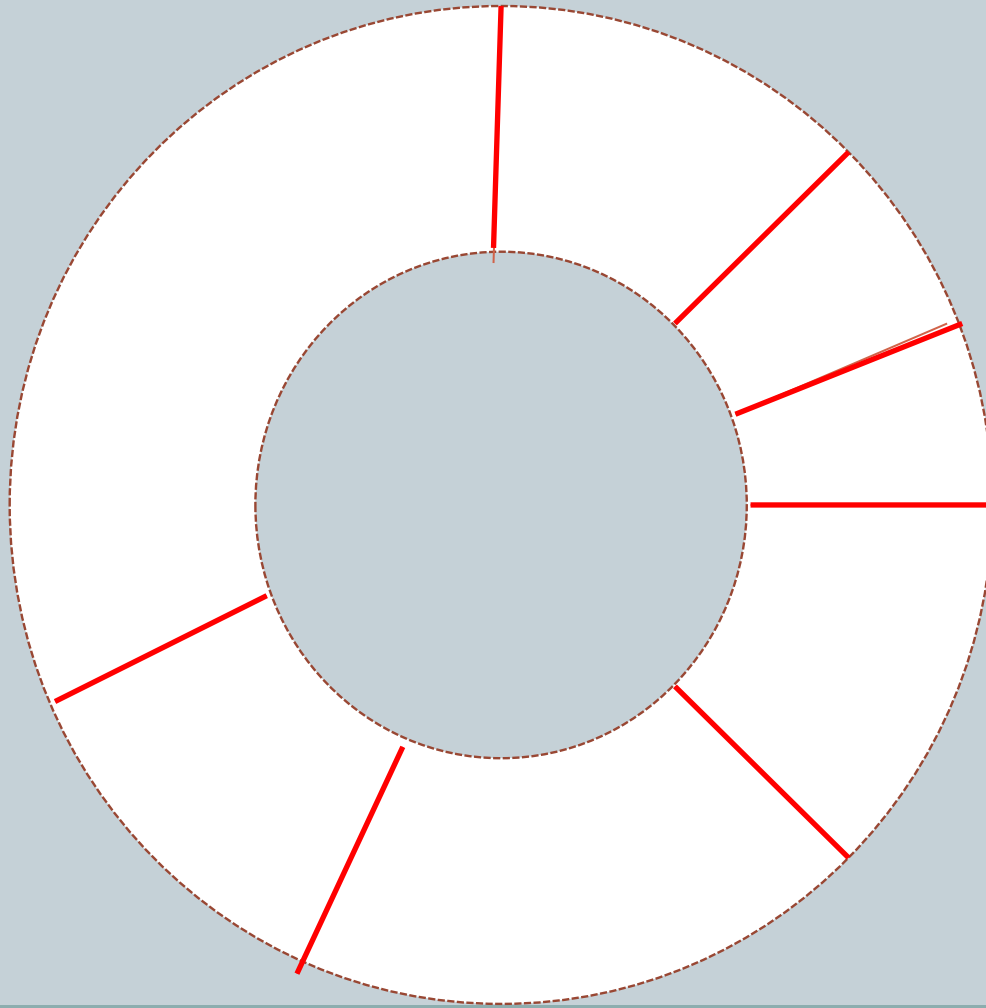


二分探索を使わない方法



- 尺取り法だけで解きたい。
- さっきの方法だと3つ目の切る位置が単調に変化しないので尺取りできない。
- そこで3つ目の切る位置を、残りの部分をできるだけ2等分に近く切る点とする。すなわち、小さいほうのピースの大きさを最大化するところできる。
- この方法でも「ある区間について、それが最小のピースとなりうるか？」の判定ができる。
- こうすると $O(N)$ で解くことができる。

二分探索を使わない方法



Sugar Glider

JOI 2013-2014 本選4
Sugar Glider(フクロモモンガ) 解説

解説: 秀 郁未 (tozangezan)

Sugar Glider

フクロモモンガとは？



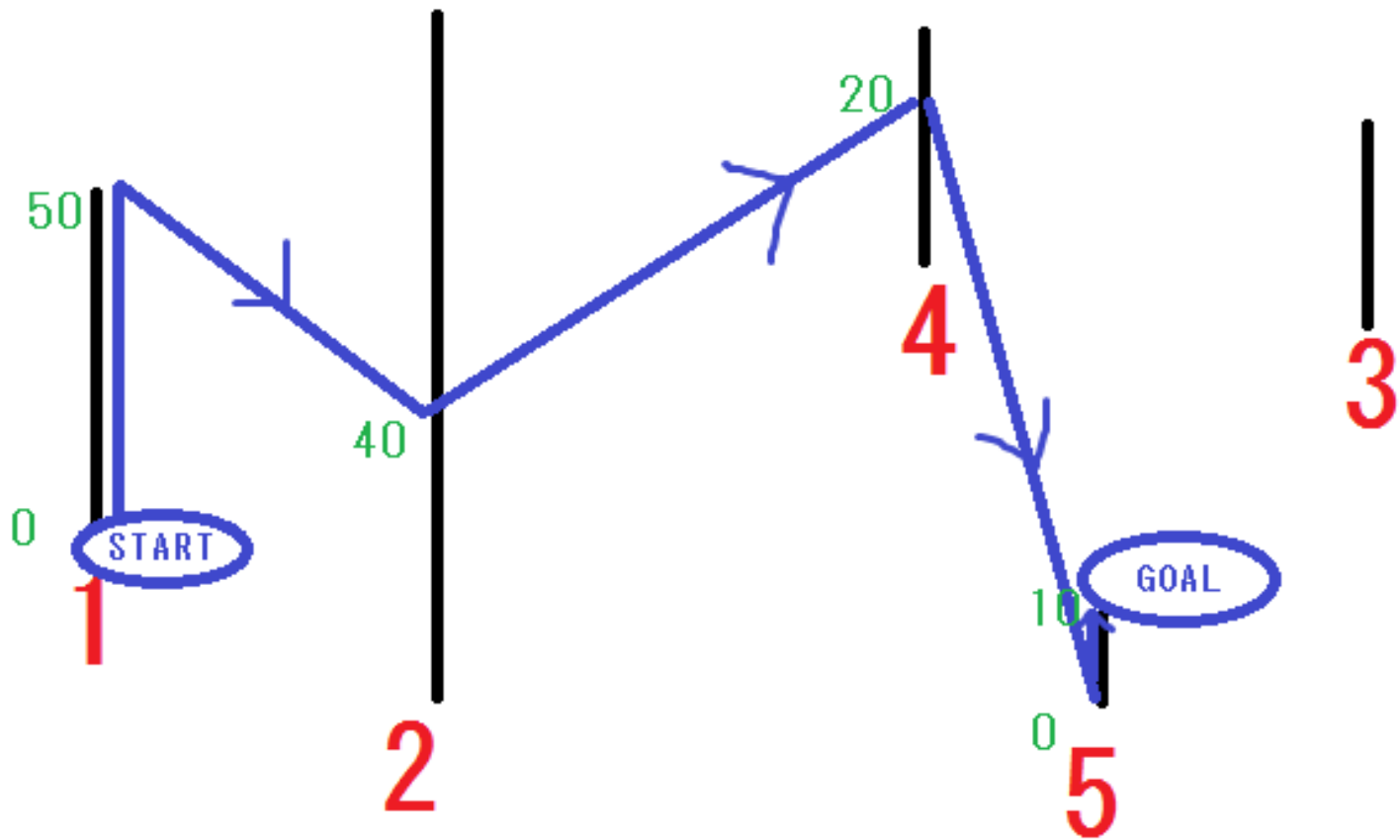
(多摩動物公園にて
撮影)

問題概要

- フクロモモンガが木の間を飛んで出発点から目的地に移動する
- 飛べる木の関係と距離は与えられる
- 木の中で高さ1上下移動するのには1秒かかる
- 木の間を飛んで移動するのには(距離)秒かかり、(距離)だけ高さが低くなる
- 飛んだ先の高さがその木の高さより高い、もしくは0未満(地中にもぐる)という移動はできない
- 最短時間はいくらだろうか？

Sugar Glider

例



最短路？

- 最小時間を求めたい
- なら最短路問題に帰着して解けそうな気がする
- いったい何を頂点としてもつのがいいのだろうか？
 - 経路によって木にいるときの高さが違う
 - 高さが違うと最短時間も違う
 - とりあえず(木の番号,そこでの高さ)の組を頂点として最短路を求めれば正解は出そう
 - 辺は高さ1だけ下,高さ1だけ上,そこから到達できる別の木の該当する高さに張る

最短路問題

- 結構いろんなアルゴリズムがある
- 今回はコストが全部正なのでDijkstra
- Dijkstra法の解説はここではしません
 - 昔より競技プログラミングが盛んになったので調べればどこでも見られます
 - 去年までの本選解説スライドにも載っています
 - 蟻本(プログラミングコンテストチャレンジブック)にも載っています
- 余談: 超密なグラフでDijkstra使うときは $O(V^2)$ にしましょう(ex. PKU2728)

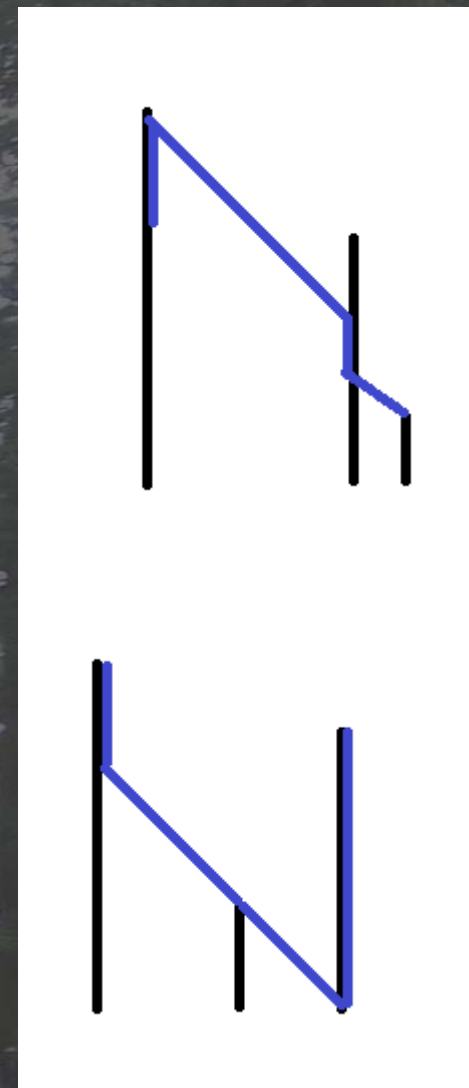
素朴な解法 計算量

- 頂点数: 木の本数*高さ = $O(\text{sum}(H))$
- 辺数: 高さ*M程度 = $O(MH)$
- Dijkstra法の計算量 = $O(|E| \log |V|)$
- 小課題1 (25点) が得られる

Sugar Glider

考察

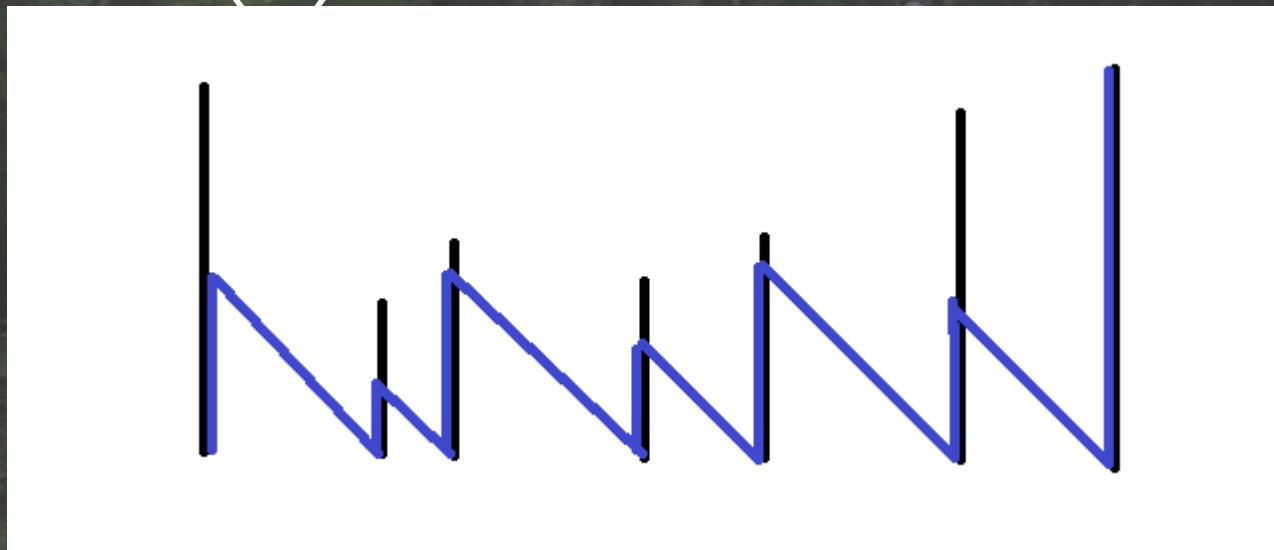
- 完全に無駄な移動...登って降りる
- つまりこういう移動 →
- 降りて登るのは有りうる →
- どういう移動が理想的なのだろうか



小課題2

- 前のページのことを考えると、
- 登りすぎないように移動すれば最適
- →(startが0なので、)移動したいぶんだけ登って、移動する(すると目的地でも高さ0)
- この登り方だけに限定して考えると、頂点数 $O(N)$, 辺数 $O(M)$ の最短路で求められる。

• 図



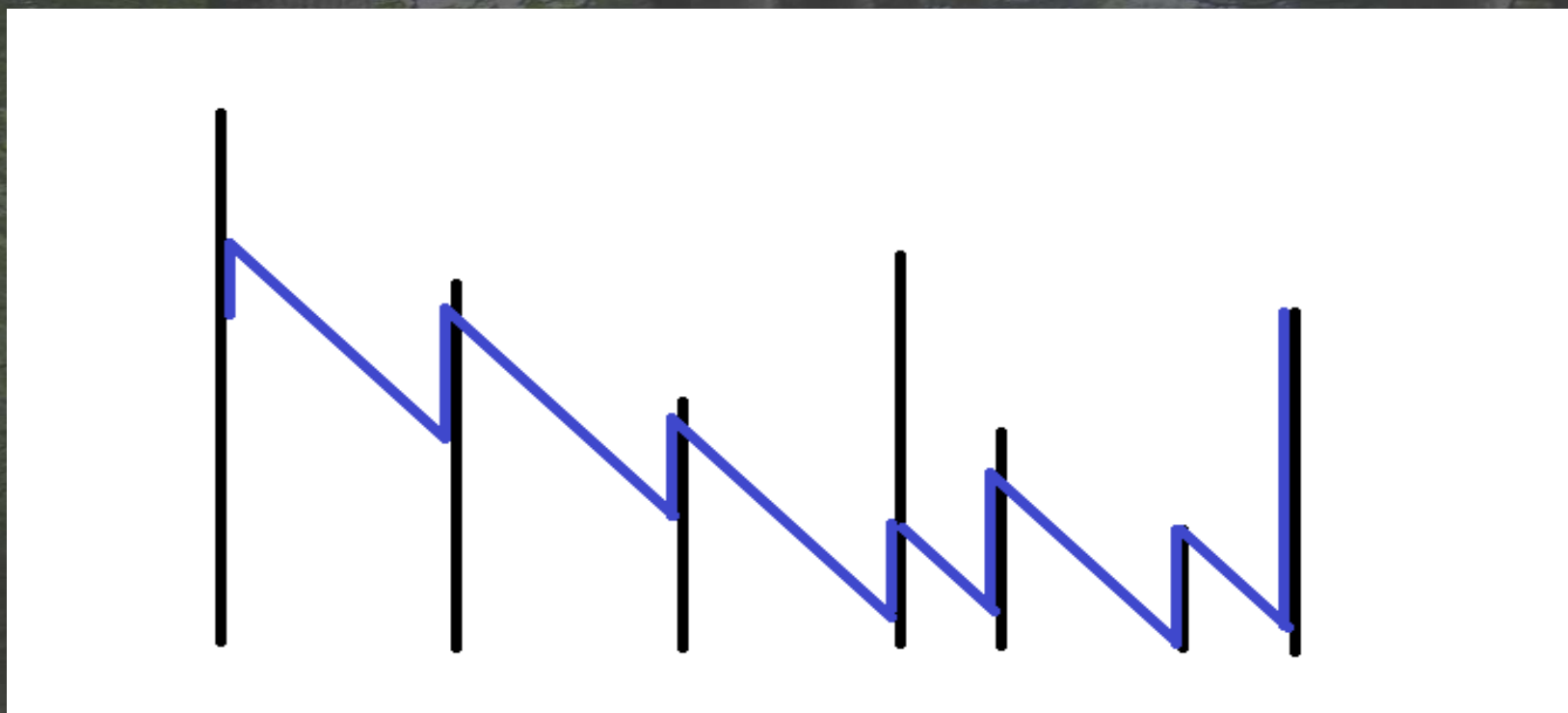
小課題2

- さっきの図を眺めてみると、
- 求める答えは、出発地から目的地までの距離(木の高さを考えない)*2+目的地の木の高さで求められる。
- ただし、絶対移動できない辺があるので、その辺は無視しないといけない。
- こういうやつ(左から右には辺を張れないが、右から左には辺を張れることに注意)
- 小課題2 (25点) が得られる



小課題3の考察

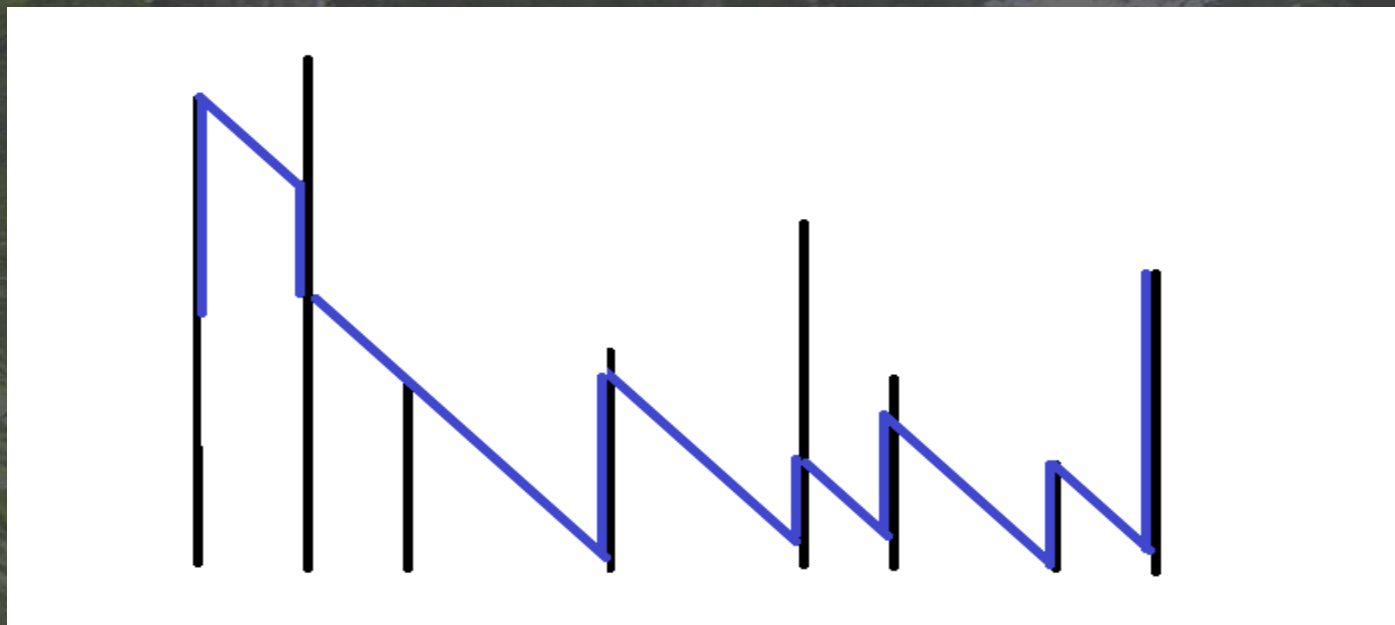
- (経路が決まったとして)その中で最適な移動の仕方を考えよう。



- 最適なパターンの一種

小課題3の考察

- 最適でない例

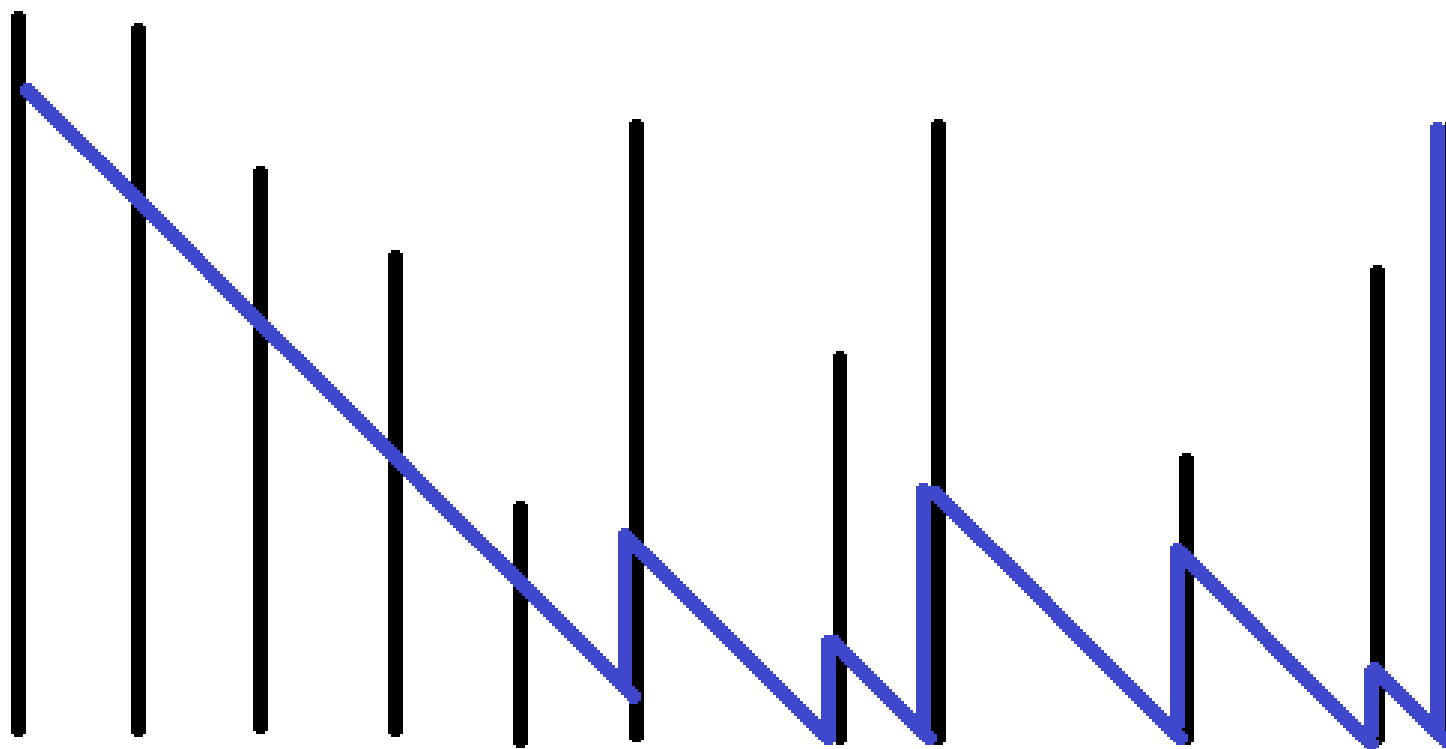


- 何枚か前のスライドに書いた「登って降りる」無駄な移動をしている
- これが起きないような移動の仕方なら最適

小課題3の考察

- さっきの無駄な移動をしないもっとも単純な移動を考えよう(なるべく単純な移動にすることで、最短路問題の頂点数を $O(N)$ にできるのか?)
- →できます
- その答えは...

小課題3の考察



小課題3の考察

- 各木でやっている移動は以下のとおり
- 移動先の高さが木の高さより高いとき
→移動先の木の高さまで移動
- 移動先の高さが0以上木の高さ以下のとき
→そのまま移動
- 移動先の高さが0未満になるとき
→移動先の高さが0になるように登って調整 (調整できないときはその移動は無理)

小課題3の考察

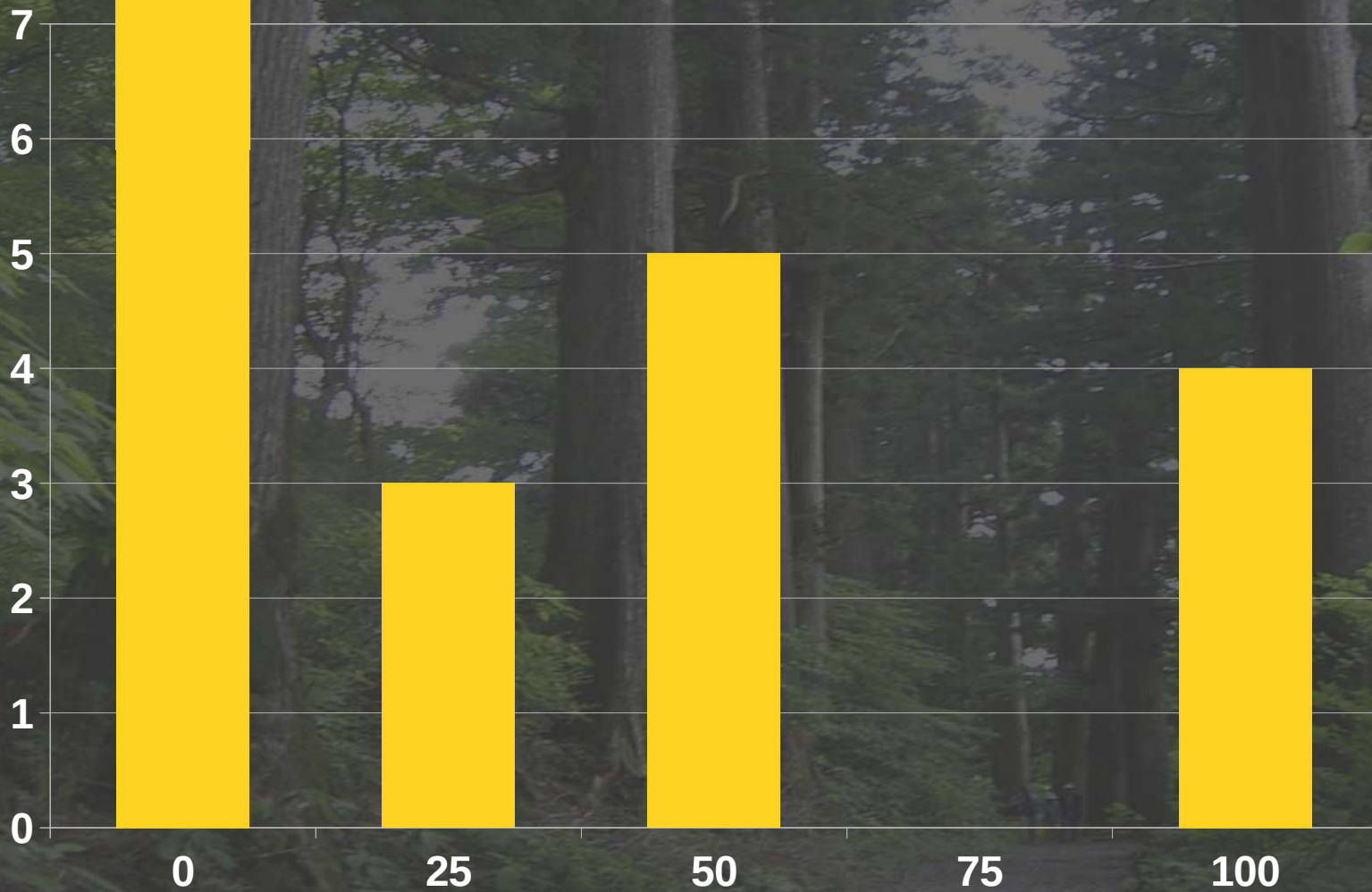
- 頂点数は $O(N)$, 辺数は $O(M)$
- 各頂点における到達までの最短時間を持っておくだけで、その木における高さは容易にわかる
- 初期位置の高さを X とすると、
- $\text{cost}[i] \leq X$ なら高さ $(X - \text{cost}[i])$ にいる
- $\text{cost}[i] > X$ なら高さ 0 にいる
- 最終的に答えは $\text{cost}[N] + \text{最後の移動}$
- 最後の移動の高さ調節も同様に計算できる

満点

- これでどういうケースであっても頂点数 $O(N)$, 辺数 $O(M)$ の最短路の計算時間で解ける。
- 到達できないときは-1を出力する
- Dijkstra法を使えば余裕な時間
- 小課題1~3 (100点) が得られる
- 移動の仕方が何種類があるのでそれらを全部もれなく書くことが大事(頑張ってください)
- あとオーバーフロー

Sugar Glider

得点分布





JOI 2013-2014 本選 切り取り線(Cutting) 解説

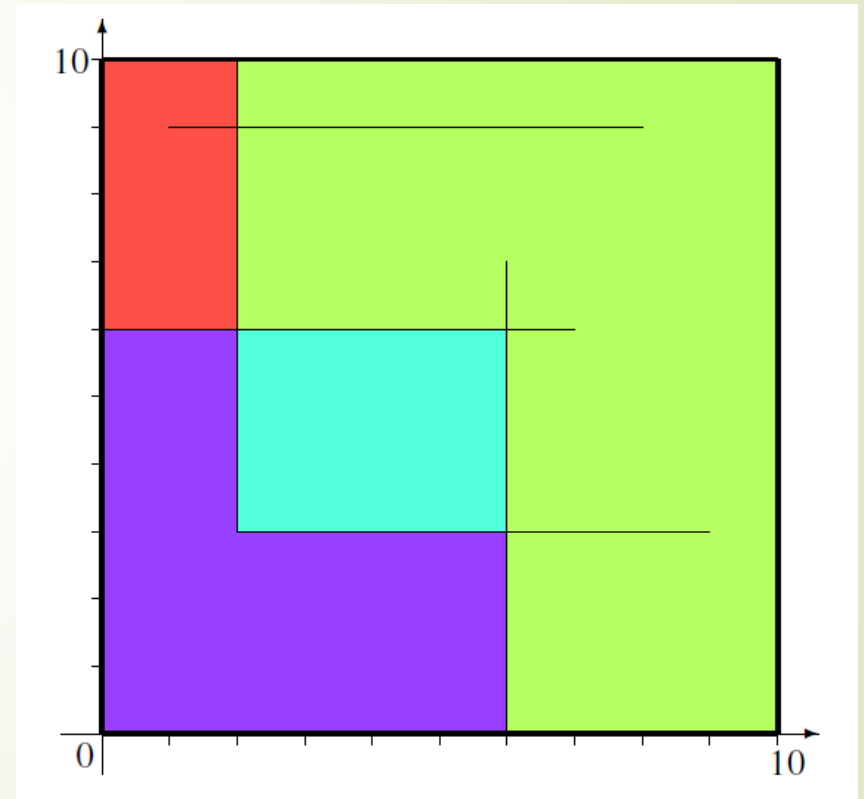


問題概要

- 長方形の紙があります
- たくさんの切り取り線に沿って切り取ります
- いくつの部分に分かれる？

入力例

- 入力例 1
- 切り分けられる部分ごとに色を塗ると右図のようになる
- 部分の中に一見余分に見えるような切り取り線があっても一向にかまわない





おことわり

- 解説の時間は限られているので、よくあるアルゴリズムについての詳細は省略します
- Segment Tree, 座標圧縮, Union Find など
- 「プログラミングコンテストチャレンジブック」
- ほか, インターネット上の資料などを参照してください

探索による解法

- 小課題 1
- 紙がとても小さく, 切り取り線も少ない
- 1×1 のマス目が $W \times H$ 個並んでいると思ってまだ大丈夫
- 隣り合うマス目の間に切り取り線があるか記録する
- 紙がいくつの部分に分かれるか探索
 - 幅優先探索 (BFS) / 深さ優先探索 (DFS)
- $O(WH + N(W+H))$
- がんばって探索を書くと小課題 1 が解ける (5 点)

座標圧縮

- 小課題 2
- 切り取り線は 1000 個以下だが、座標がいたずらに大きい
- 座標圧縮しましょう
- 覚えるべき座標は、紙の端と、切り取り線の座標で 1 回以上現れるものだけ
- X, Y 座標ごとに高々 $2N + 2$ 個
- $O(N^2)$
- 座標圧縮して探索すれば小課題 1, 2 が解ける (10 点)



探索の限界

- 小課題 3, 4, 5
- もはや切り取り線は 100000 本も存在する
- 普通に探索しようとしたらそもそもメモリが足りない

「長方形の紙」の除去

- 紙の外に切り取り線はない
- 紙の辺を特別扱いするのは面倒
- 無限に広い平面を，紙の辺と切り取り線たちで分割する問題に置き換える
 - すると，分割数 $- 1$ (長方形の外を除外) が答えになる

領域の分割 -> 領域の併合

- 間違っても、分割された領域の形なんて考えたくない！
 - サンプルの「J」「O」「I」どころでない大変な形が出てくる
- 領域の分割は大変
- 逆に、領域をくっつけるのはそれ自体は簡単
 - 今は、領域の数にしか興味がない
 - Union Find が使える
- まず、領域を激しくぶった切って、あとでくっつけよう

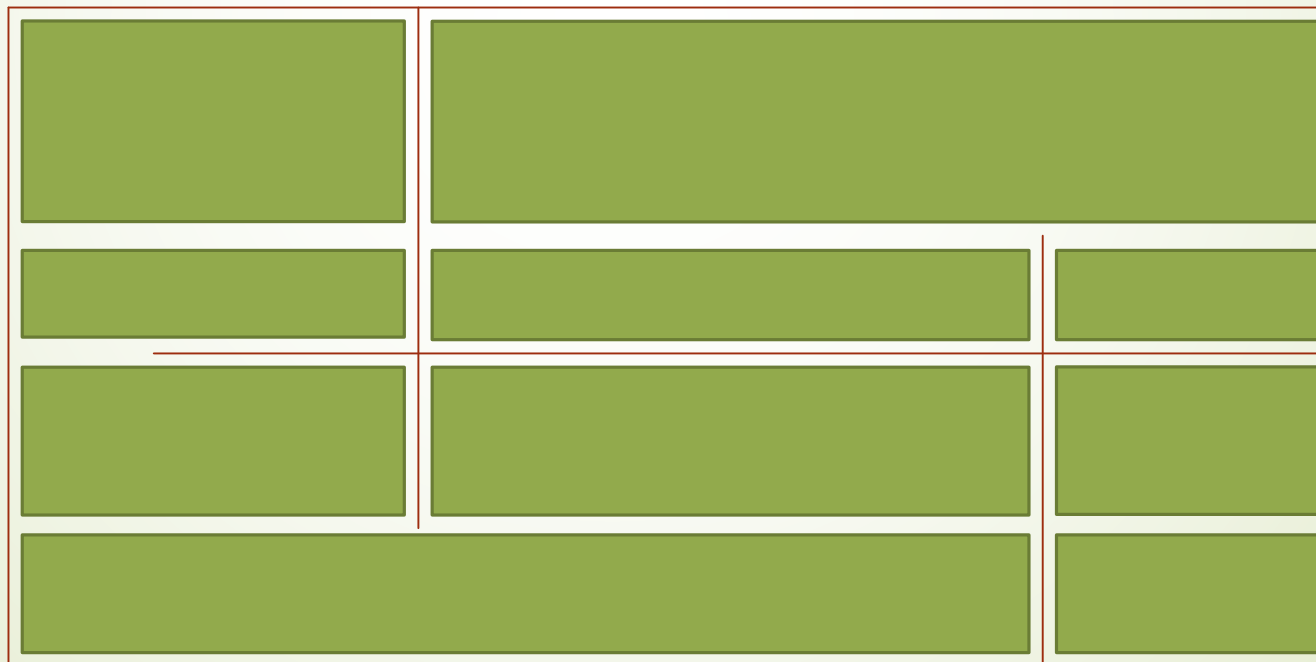
領域をバラバラにする

- 長方形なら扱いやすい
- 領域を，長方形に分割して考えよう
- でも，さすがに全部 1×1 とかはあまりうれしくない

領域の分け方

- たとえば...

- Y 座標については, 全部すべての場所で区切る
- Y 座標ごとには, X 座標は必要な部分だけで区切る



領域併合による自明な解法

- さっきみたいに予め領域をたくさんの部分にまず分割する
- 領域ごとに Union Find のノードを持たせる
- 隣り合っている場所をすべて調べて、くっつける
 - 幸い、Y 方向についてのみ見ればよい
 - X 方向は分割の段階で全部くっつけておいた
- $O(N^2)$ から何もよくなっていない！

観察

- Y 座標ごとに, X 方向でどう分割されてるか眺める
- Y 座標が 1 動いただけでは分割の様子はほとんど変わらない
 - 分割の様子が変わるのは, Y 方向の線分が出たり消えたりするとき
- しかも Y 座標が 1 動いたくらいで領域を別扱いする必要もあまりない
 - X 方向の線分が邪魔しなければ同じ領域

平面走査へ帰着

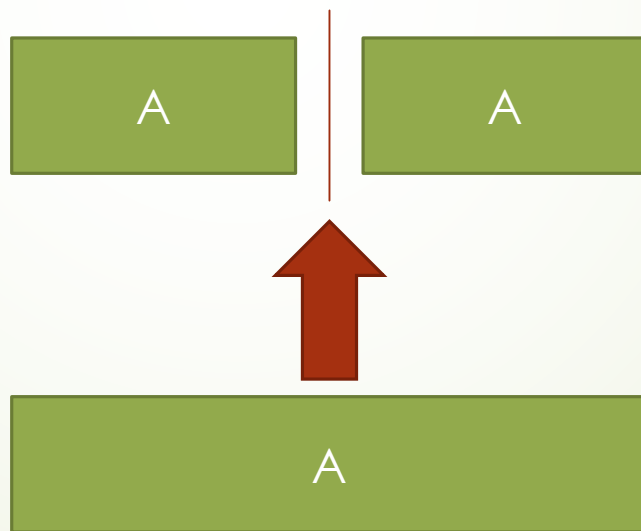
- X 方向の分割の状態を持って, Y 座標を $-\infty \rightarrow \infty$ と動かして処理する
 - 分割の状態は, 適切なデータ構造を使う
- 動かしてる途中に, 線分が出てきたらうまく対処する
- たとえば, 「線が出てくる」「線が消える」「X 方向の線」などのイベントを並べておいて, ソートして最初から見る
- X 座標の状態をデータ構造に持って, Y 座標を動かすっていうのは頻出手法です
 - 2013 春合宿 Construction
 - 2012 春合宿 Fortune Telling など...

データ構造

- 各領域の左端, 右端くらいは覚えてほしい
- Union Find のノードも覚えてほしい
- たとえば, set に (left, right, node) たちを放り込む
 - right は次(右)の領域の left なので特に覚えなくてもよい
 - set なので検索が $O(\log N)$ ができる

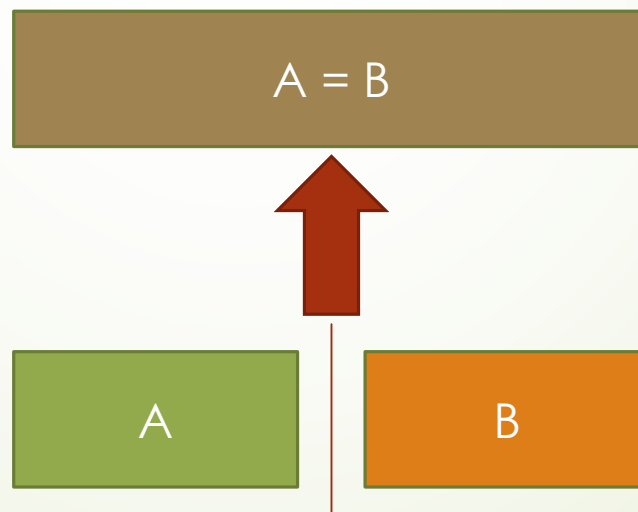
線分イベント(1): Y 方向線分の出現

- 領域は分割されます
- 分割後の 2 つの領域の Union Find のノードは両方とも分割前と同じ



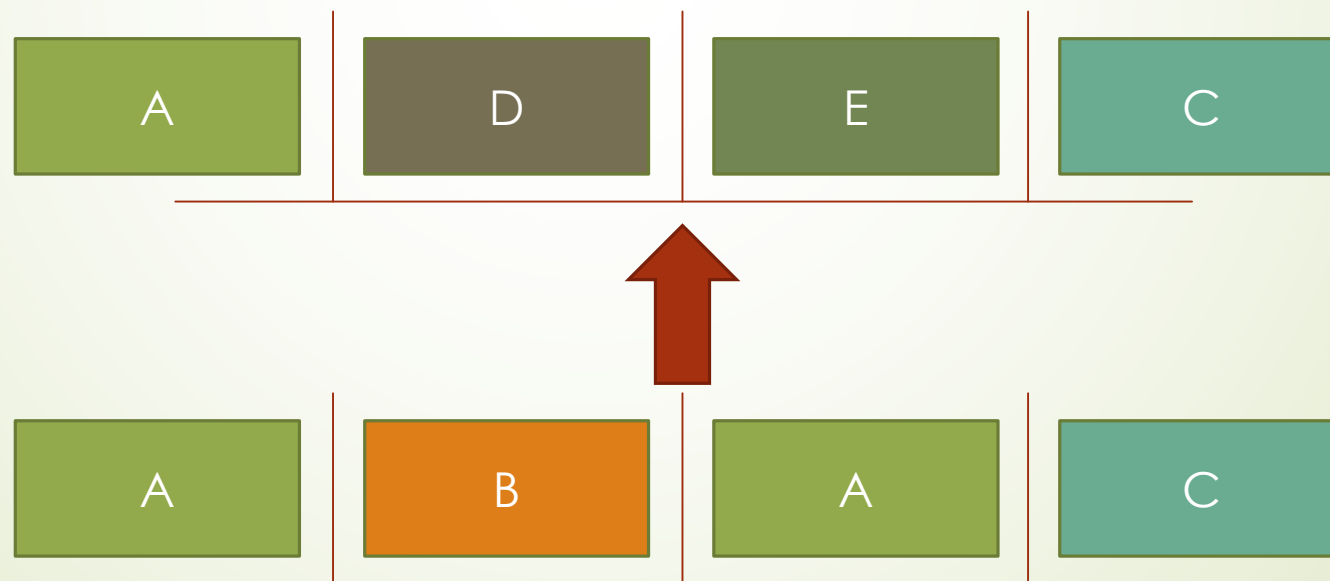
線分イベント(2): Y 方向線分の消滅

- 領域はくっつきます
- 元々の 2 つの領域の Union Find ノードは merge される
- 新たな領域に対応するノードは, 今 merge したノード



線分イベント(3): X 方向線分

- 領域の状態に変化はありません
- が, Union Find ノードが新しくなります
- その線分によって完全に覆われる領域たちすべてのノードを新品にする
 - 古いものは, その領域に関わっていたことは忘れてしまう



データ構造に対する効率よい処理

- set を使ったとします
 - 自分で Segment Tree を書く場合は適切に機能を実装
- Y 方向線分が出てきたとき, それがどの領域に関わるかは $O(\log N)$ でわかる
- 領域の追加/削除も $O(\log N)$
- Union Find の時間は定数みたいなもの
- X 方向線分については, 「どこからどこまで更新する必要があるか」までは $O(\log N)$
- その更新が大変
 - だいたい, 線分たちの交点の数に比例する時間
- それでも, ここまでで小課題 3 は解けて, 全部で 30 点

ボトルネック

- 「その線分によって完全に覆われる領域たちすべてのノードを新品にする」
- これがとんでもないネック $O(N^2 \log N)$ の元凶
- 困ったことに、関係するノードは全部新品にしないといけない
 - 後でどこが使われるかわからない！

遅延処理

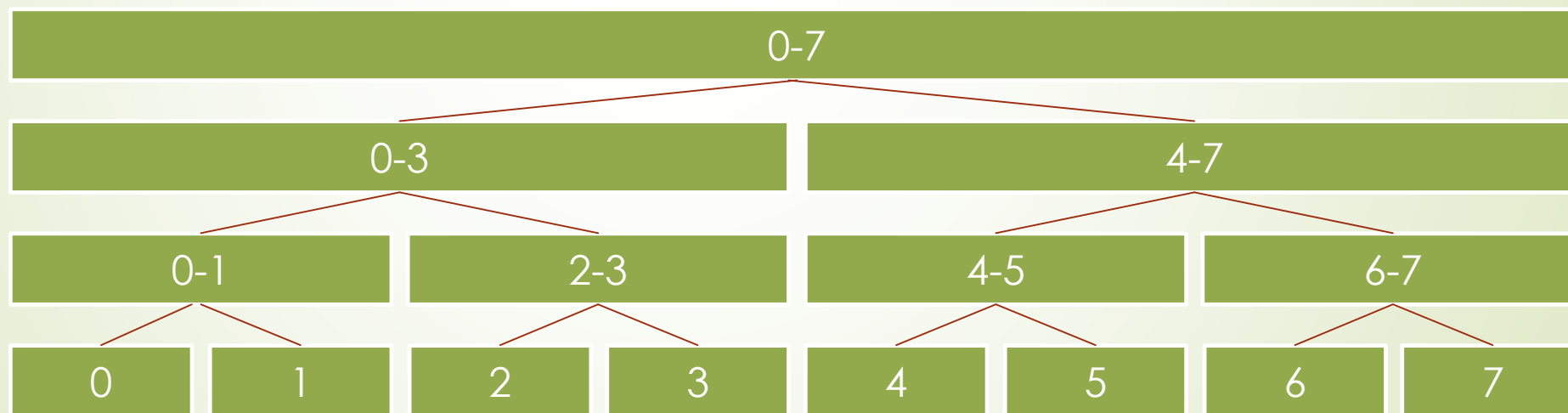
- 逆転の発想！！！！
- 「ここらへんは新品」という情報だけ覚えておく
 - 新品は、使う前に新しいノードに更新する
- 範囲に対して操作をする Segment Tree でたまに使う手法
- 今回は、Segment Tree を媒体にノードの生成を遅延させて制御する

必要な処理

- 1. ある範囲のノードにすべて新品というフラグを立てる
- 2. ある位置のノードを新品でなくする
- ノードを読み出すときは、フラグを確認して新品だったら使う前に新しいものに変えておく
- 書き込むときは、ノードを新品でなくしてから書き込む

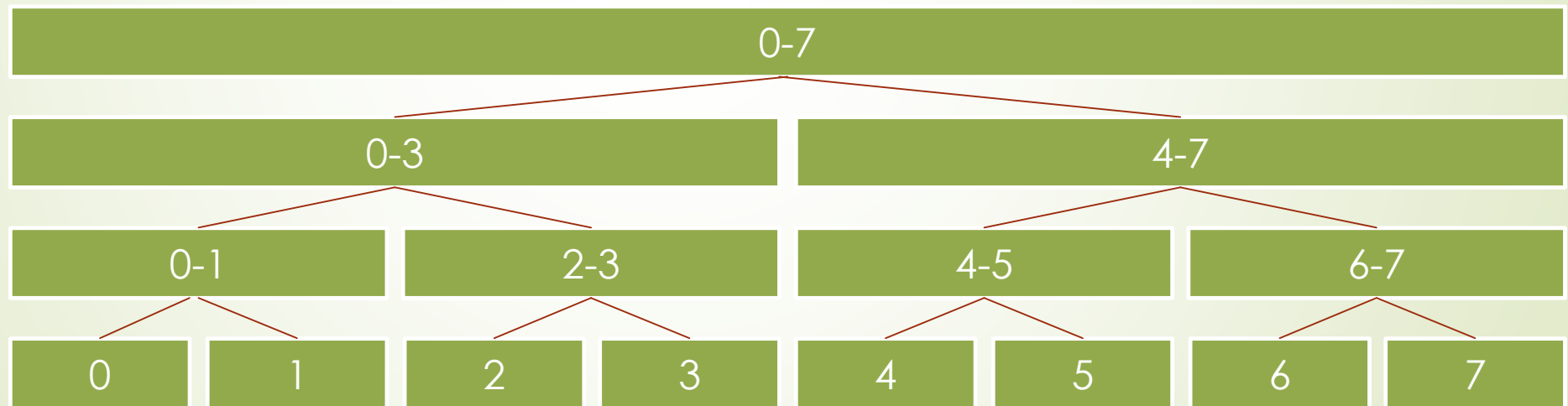
Segment Tree の作り方

- 構造は普通のもので同じで、ツリーの節に範囲の情報を持たせる
- 節にフラグが立っていたら、その範囲全部新品



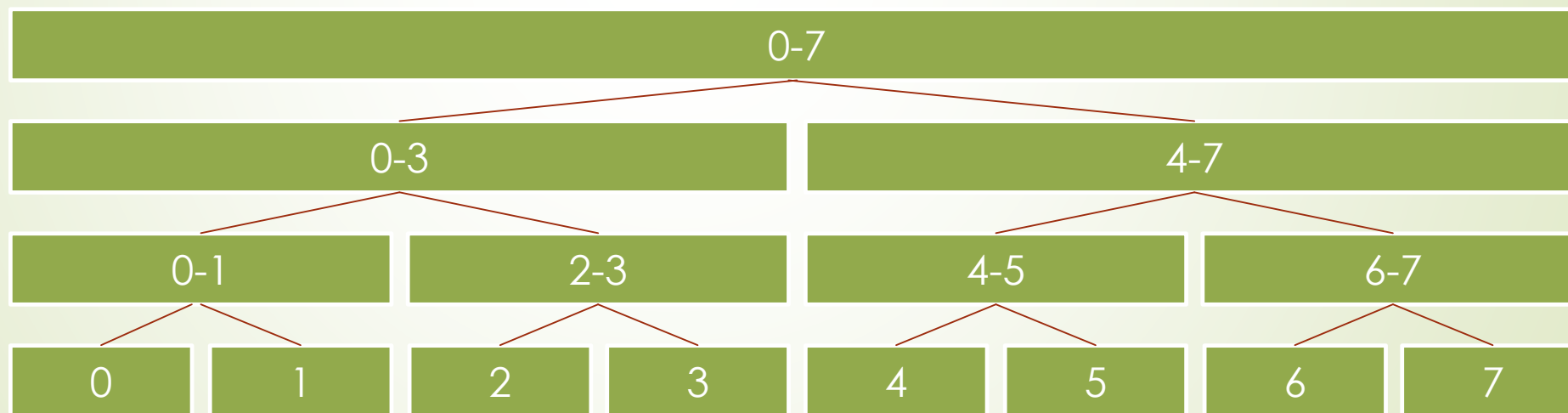
Segment Tree の更新 (1)

- ある範囲にフラグを立てるとき
- 一番上から節を見て行って、今の節の範囲が操作範囲に全部含まれていたらフラグを立て、全くかぶってなかったらやめ、微妙にかぶってる場合は下の両方の節に対して操作する



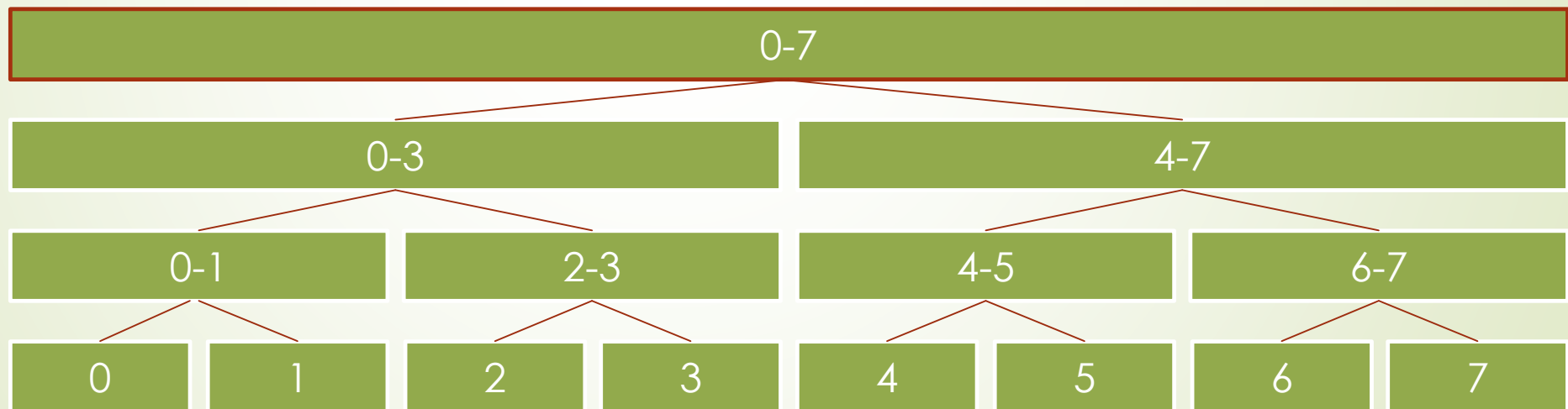
Segment Tree の更新 (2)

- 2, 3, 4, 5, 6, 7 に対してフラグを立てる場合の例



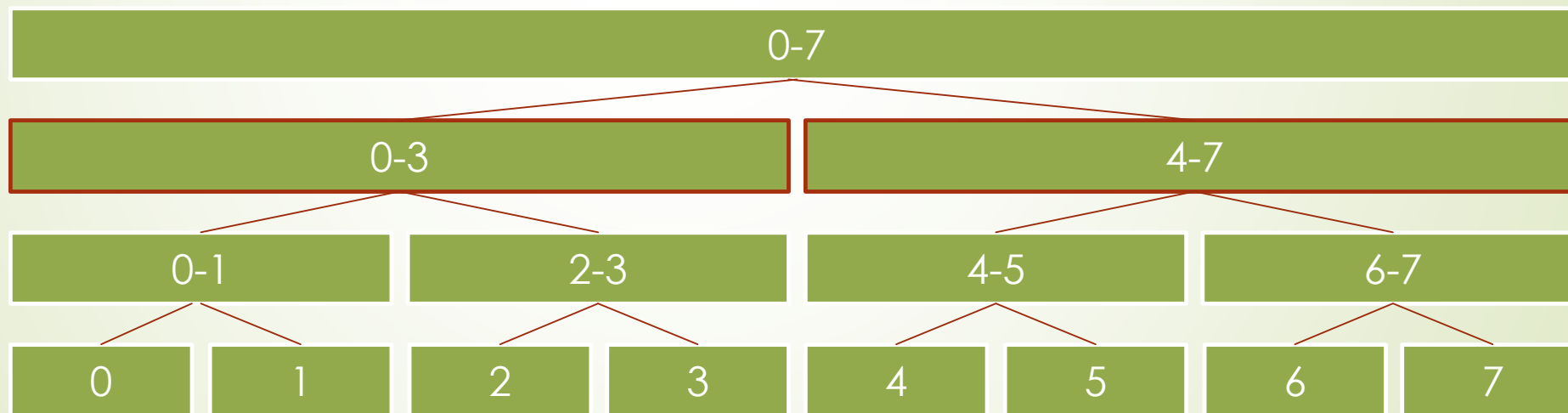
Segment Tree の更新 (2)

- 2, 3, 4, 5, 6, 7 に対してフラグを立てる場合の例



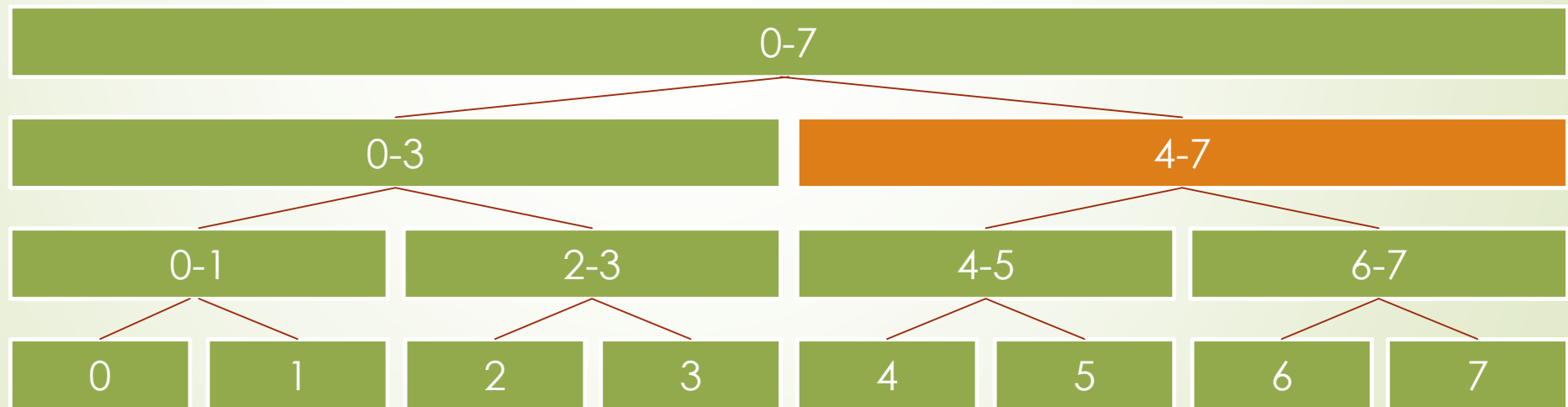
Segment Tree の更新 (2)

- 2, 3, 4, 5, 6, 7 に対してフラグを立てる場合の例



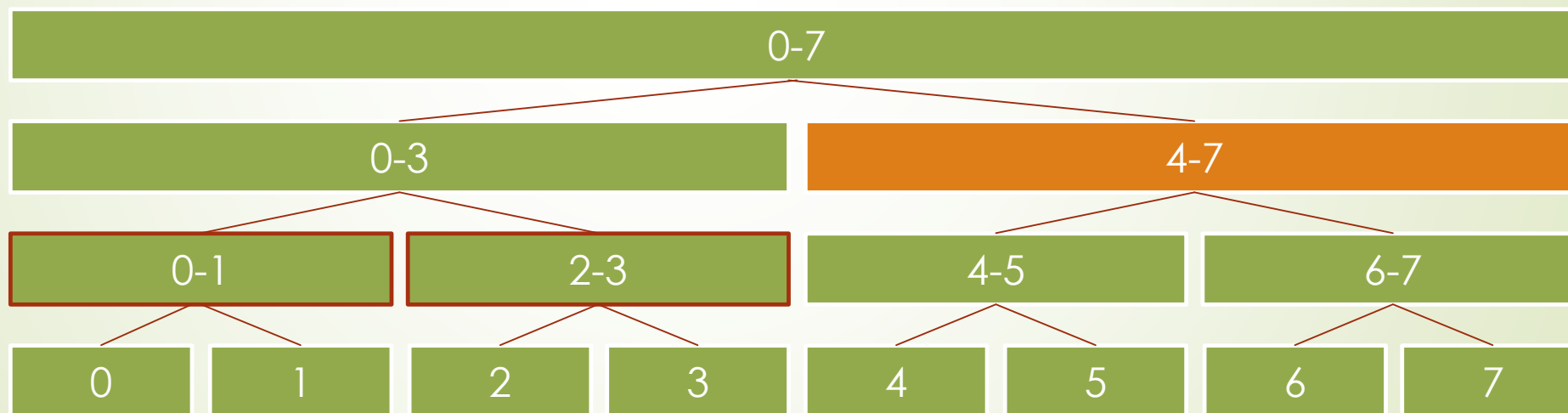
Segment Tree の更新 (2)

- 2, 3, 4, 5, 6, 7 に対してフラグを立てる場合の例



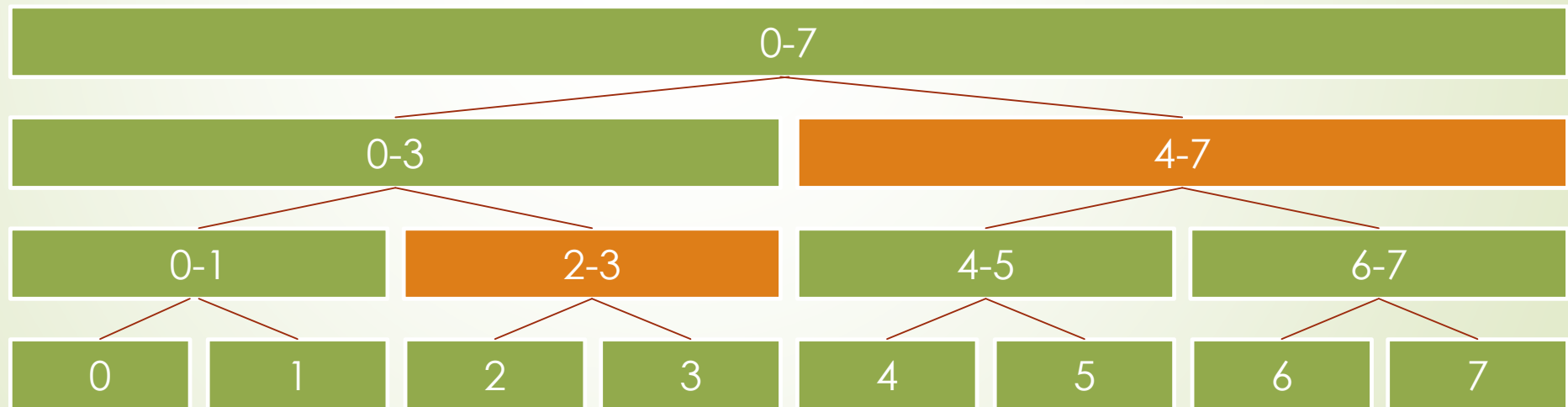
Segment Tree の更新 (2)

- 2, 3, 4, 5, 6, 7 に対してフラグを立てる場合の例



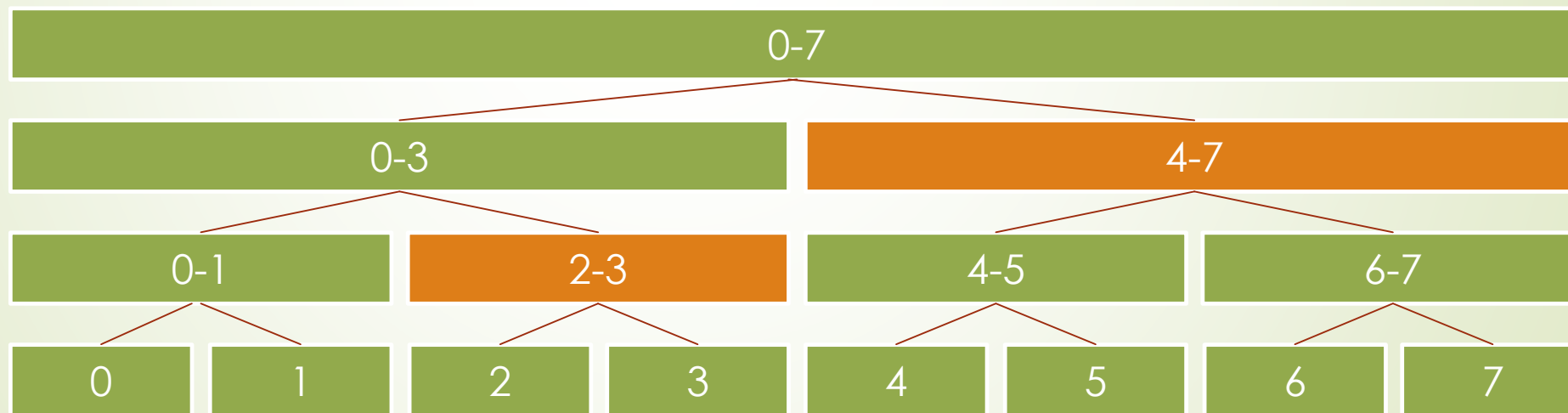
Segment Tree の更新 (2)

- 2, 3, 4, 5, 6, 7 に対してフラグを立てる場合の例



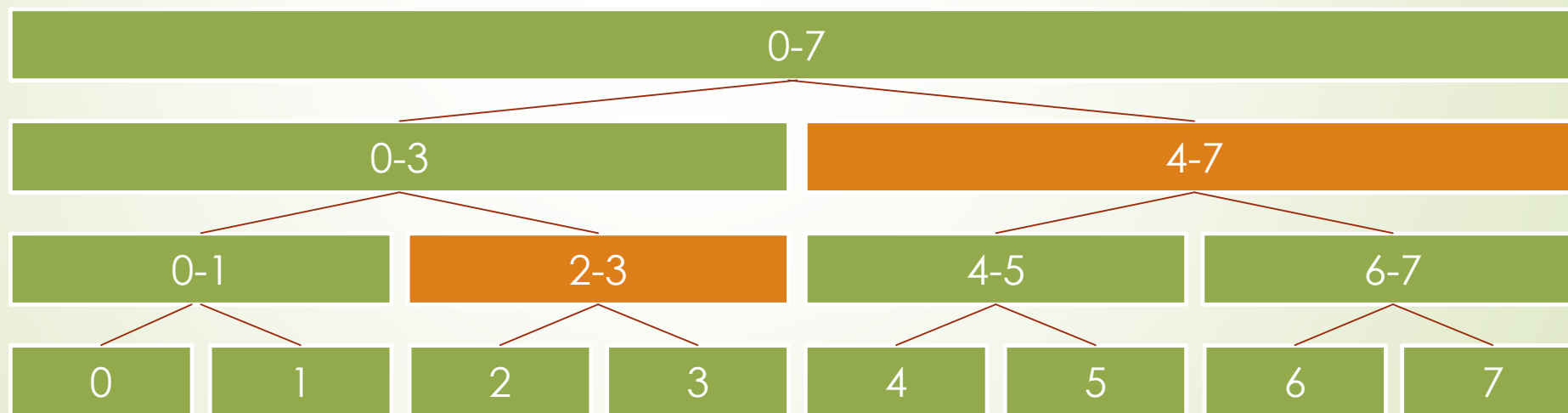
Segment Tree の更新 (3)

- ある位置のフラグを取り除くとき
- その位置を含む節たちを上から見ていって、フラグが立っている節があったら消し、左右の子にフラグを立てて、次の節を見る



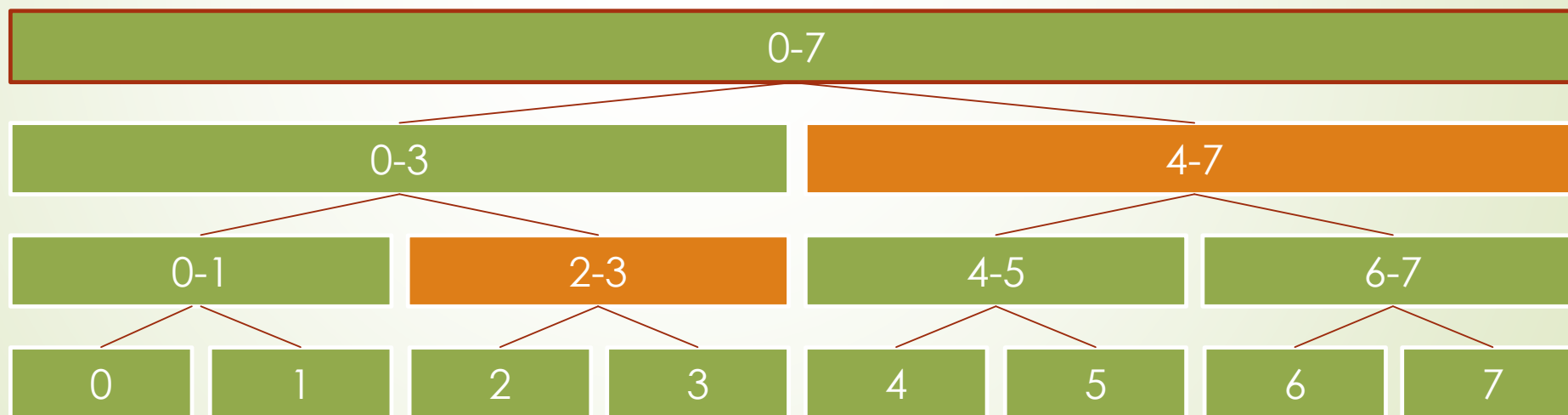
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



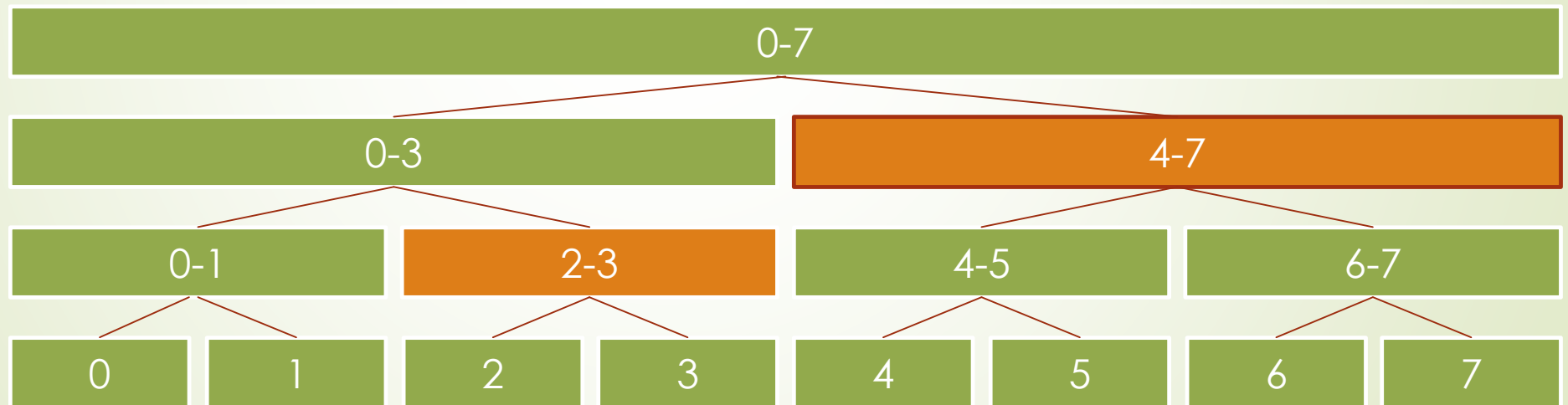
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



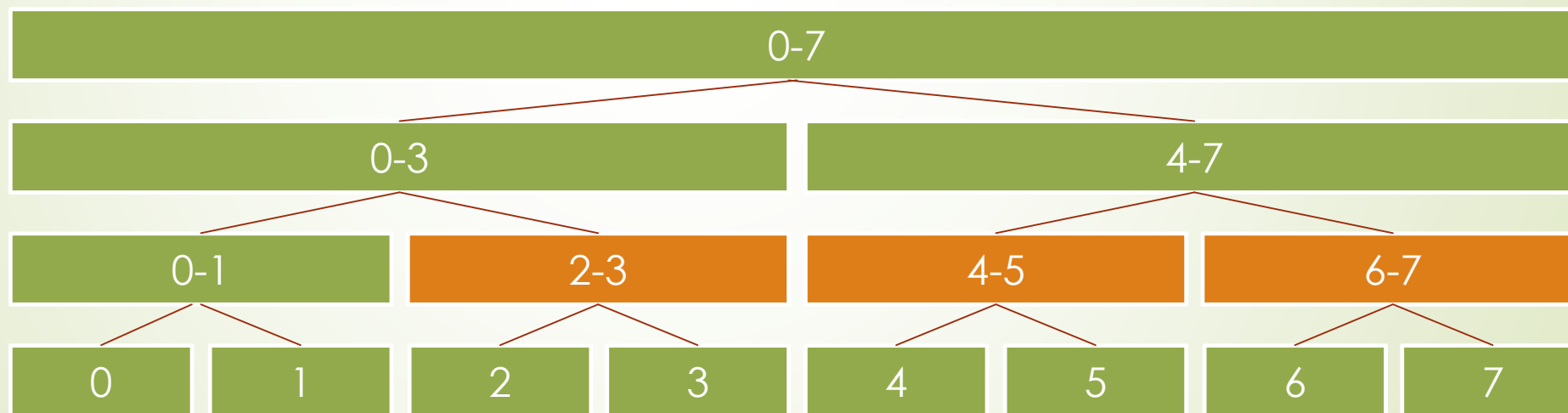
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



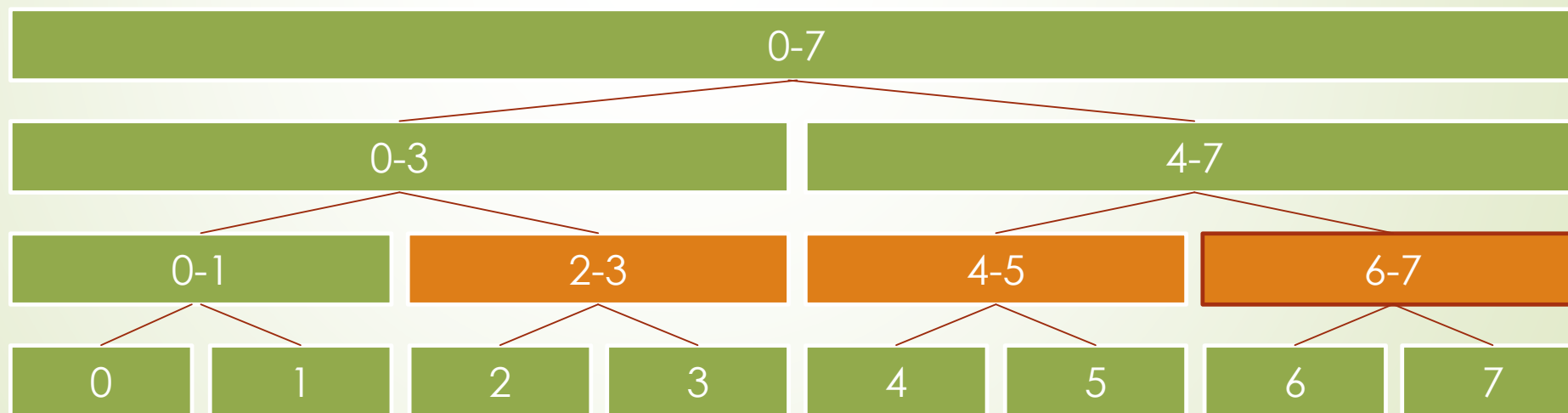
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



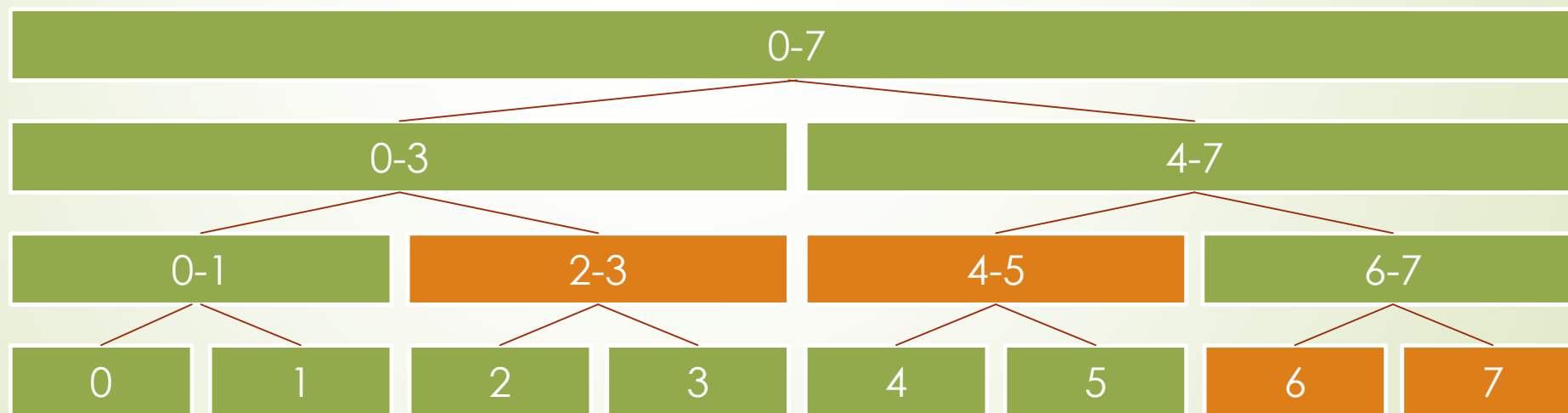
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



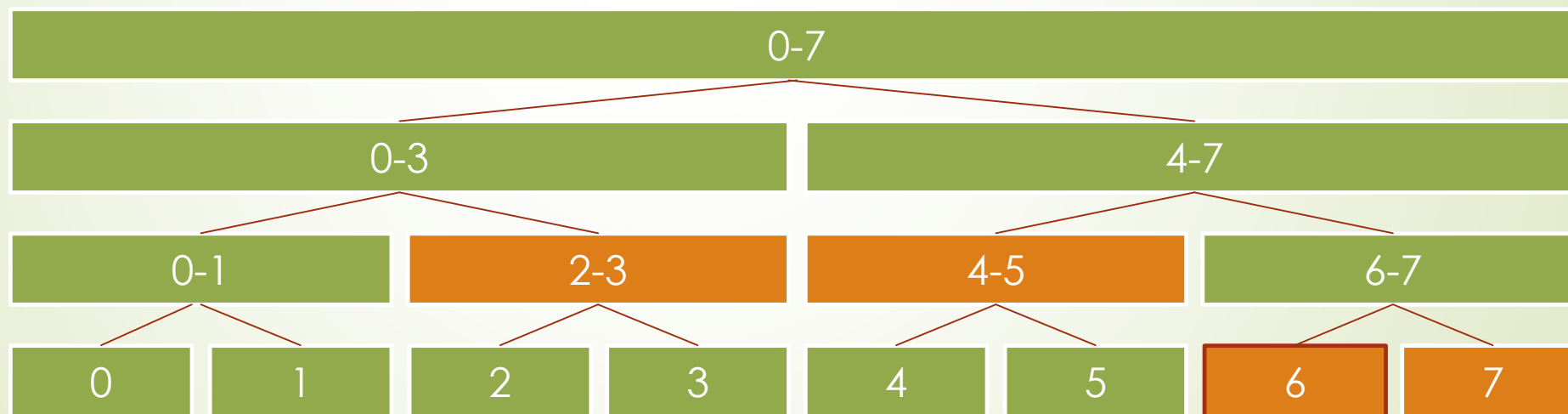
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



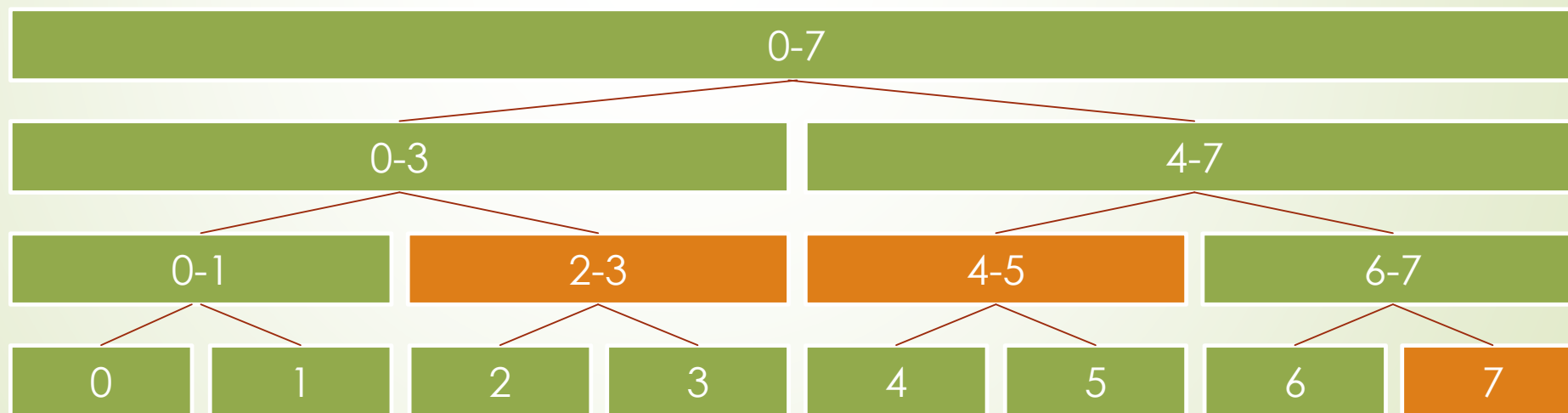
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



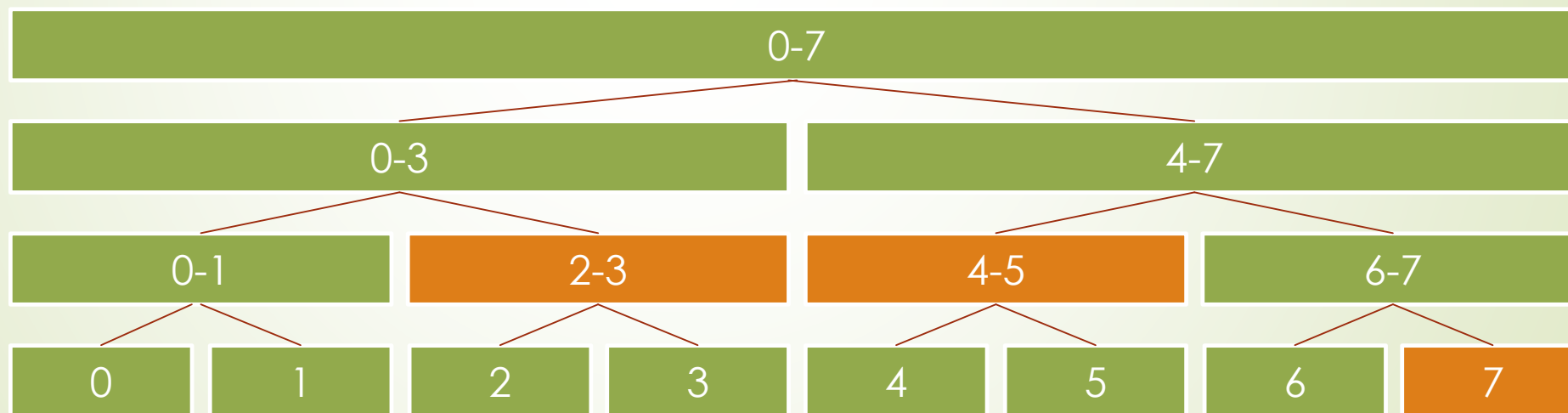
Segment Tree の更新 (4)

- 6 のフラグを取り除く場合の例



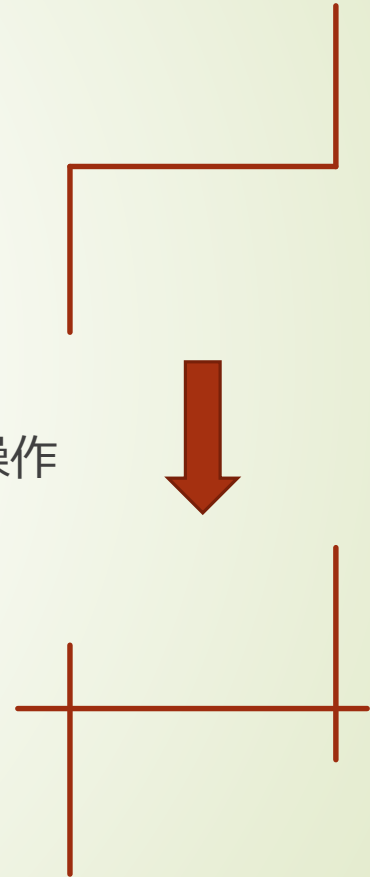
Segment Tree の参照

- ある位置にフラグが立っているか知りたいとき
- その位置を含む節たちをすべて調べて、1 つでもフラグが立っていればフラグが立っている



諸注意

- 線分が端点で交わってる場合
- 少しだけ伸ばしましょう
- あるいは、同じ y 座標に出てきたとき適切な順番で操作
- Y 方向線分出現→X 方向線分→Y 方向線分消滅、の順で操作
- 同じ種類のものが同じ位置にある場合はどの順番で操作してもよいです





まとめ

- この Segment Tree を使って, Union Find のノードの生成を遅らせる
- 操作の回数は $O(N)$
- それぞれの操作は $O(\log N)$
- 全部で $O(N \log N)$

- これで 100 点



別解

- 「長方形の紙を除去」のところから分岐
- 分割数を別な方法で求められないか？

観察 (1)

- 平面にでたらめに線分（切り取り線）をたくさん描いてみて観察
- 領域の分割数，線分の数，線分の交点数，線分の連結成分数に注目
- 何もないうきは領域は 1 個に分かれる

観察 (2)

- 線分を描いたが、その線分が何とも交わっていないとき
- 領域の数は変わらない
- 辺の数は 1 増える、線分の連結成分は 1 増える、交点は増えない



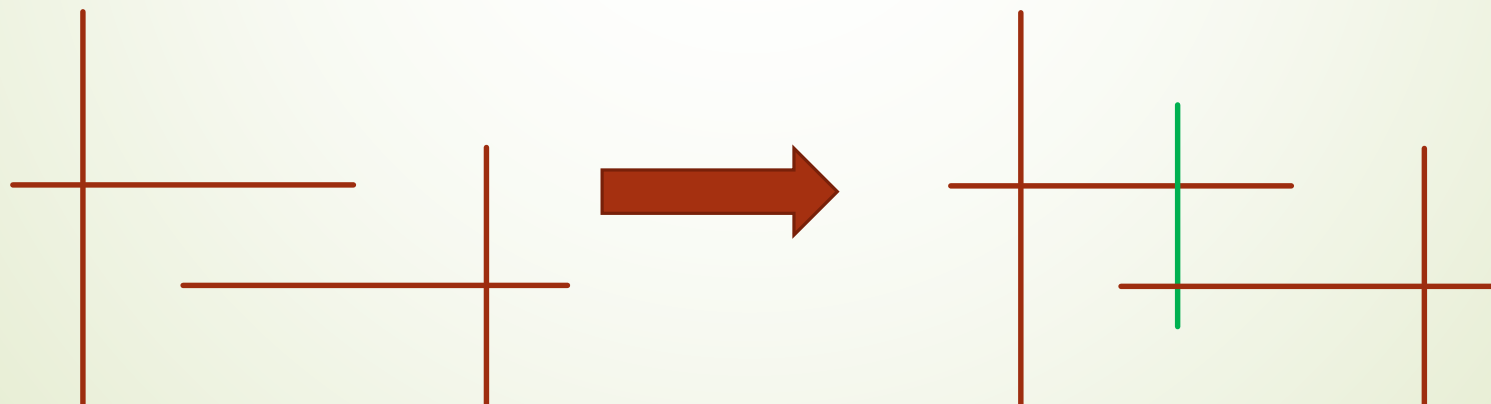
観察 (3)

- 他の線分との交点が 1 個しかない線分を描いたとき
- 領域の数は変わらない
- 辺の数は 1 増える, 線分の連結成分は増えない, 交点は 1 増える



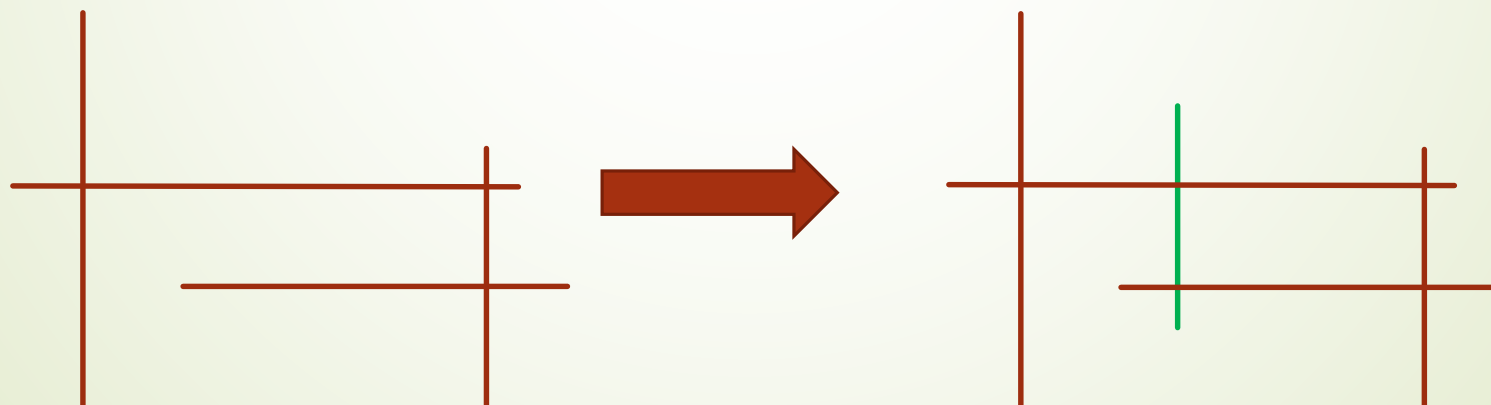
観察 (4)

- 2つの異なる連結成分を結ぶように線分を描いたとき
- 領域の数は変わらない
- 辺の数は1増える, 線分の連結成分は1減る, 交点は2増える



観察 (5)

- 同じ連結成分内を結ぶように線分を描いたとき
- 領域の数は 1 増える
- 辺の数は 1 増える, 線分の連結成分は変わらない, 交点は 2 増える



観察 (6)

- 表にしてみます

領域の数	交点の数	線分の数	連結成分数
± 0	± 0	+1	+1
± 0	+1	+1	± 0
± 0	+2	+1	-1
+1	+2	+1	± 0

- $(\text{交点の数}) + (\text{連結成分数}) - (\text{領域の数}) - (\text{線分の数}) = \text{一定}$, が成り立っているように見える
- その定数は -1

Magical Formula

- 結局、領域の分割数は、
$$(\text{分割数}) = (\text{交点の数}) - (\text{切り取り線の数}) + (\text{切り取り線の連結成分数}) + 1$$

で求められる
- 「Euler の定理」と呼ばれています
- 交点の数は、Segment Tree を使って平面走査を行えば求められる
 - 詳細は省略します（最初の解の方法とだいたい同じ）
- 切り取り線の連結成分の数が問題

小課題 3, 4

- 小課題 4 はもう解けてます
 - 条件に, 連結成分の数が 1 と書いてある
 - これで 20 点
- 小課題 3 もほとんど解けたも同然
 - 平面走査を行うときに, 交差しているものを Union Find でまとめる
 - 別な 20 点
- 小課題 3 で解けなさそうなら小課題 4 で解く, とかすると両方の点を得られる
- ここまでで, 50 点

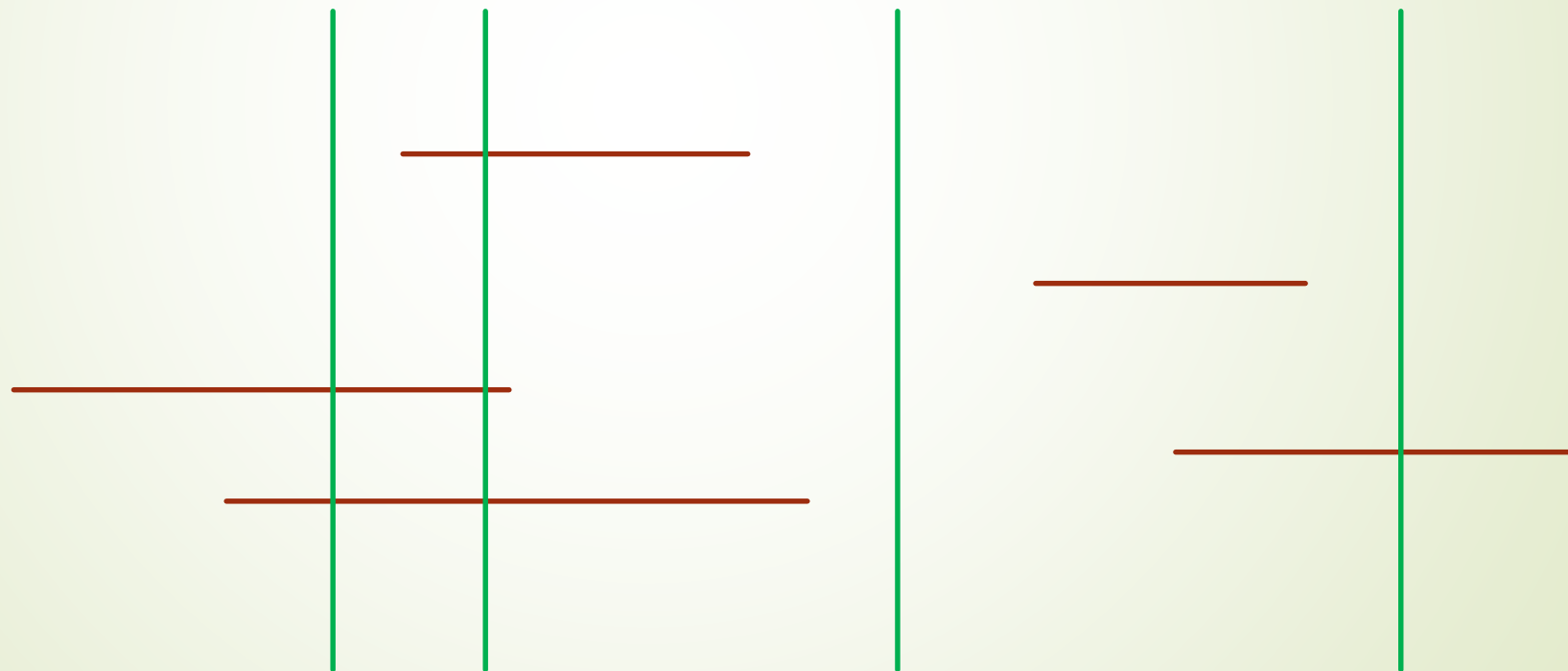


満点に向けて

- 切り取り線の連結成分の数を高速に求めないといけない
- 交点の数は最大 $O(N^2)$ になりうるので普通に求めるわけにはいかない

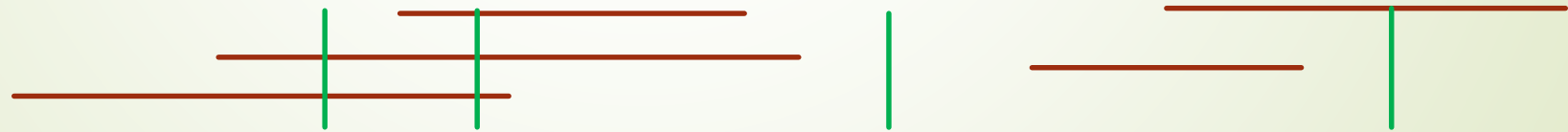
極端な場合の考察 (1)

- Y 方向の線分がすべて $(-\infty, \infty)$ を占める (無限に長い) 場合



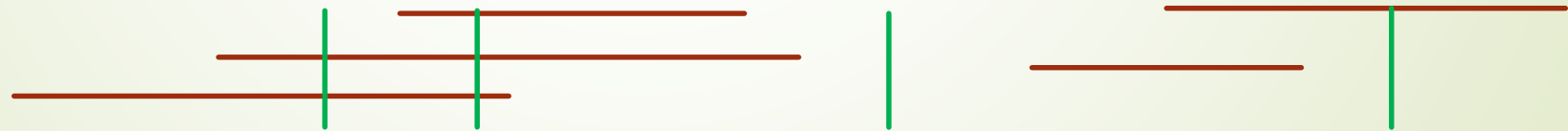
極端な場合の考察 (2)

- Y 座標はもう忘れてよい
- Y 方向の線分(直線?)は, そこにある X 方向線分たち同士(およびその線分)をつなげる働きをする
- この上で, 連結成分の数を求めたい



極端な場合の考察 (3)

- X 座標が小さいほうから見ていく
 - X 方向線分が現れたり消えたりしたらその都度操作
 - Y 方向線分が現れたら, その線分および今ある X 方向線分をすべて同じ連結成分にする
- これも最悪 $O(N^2)$ 回の併合操作が必要



極端な場合の考察 (4)

- 意味のない併合操作を何回も行うことになる
- 少し考えると、併合操作を行った後も、今ある X 方向線分を全部覚える必要はない！
 - 一番最後まで残るものだけ残して捨ててしまってもよい
- これを行うと、併合操作の回数は $O(N)$ 回になる
- あらかじめ線分の出現位置でソートを行うので、 $O(N \log N)$

線分の分解

- Segment Tree の要領で Y 方向の線分たちをぶった切る
- 切断された断片が取りうる範囲は $O(N)$ 種類ある
- 各線分は $O(\log N)$ 個に分断される



X 方向線分の取り扱い

- X 方向線分は、その Y 座標を含んでいる範囲すべてに割り当てる
- その上で、各範囲に割り当てられている線分たちを併合処理する
- 同じ「範囲」内では、X 座標さえ重なっていれば併合が行える
 - さっきの「極端な場合」の解法が使える
- これで、交わる線分たちはすべて併合される

計算量解析

- 線分はそれぞれ $O(\log N)$ 個に増える $\rightarrow$ 全部で $O(N \log N)$ 個
- 各 Y 座標範囲について (K 個のものが入っている範囲の場合)
 - ソートに $O(K \log K)$: 範囲全部では最悪 $O(N \log^2 N)$
 - 順番にたどるのは $O(K \times (\text{Union Find}))$: 範囲全部では $O(N \log N)$
- 全部で $O(N \log^2 N)$, これでも通る
- 100 点

得点分布

